Color-count-preserving double-partition encodings of list and conflict-free colorings

Mauro Lucci, Graciela Nasini, Paola Tolomei, Luis Miguel Torres

---

Editorial note. Complete original paper240–2191, native graph/performance figures, unchanged bibliography.bib and original supplied model5-names.bst retained. Original cached elsarticle class/source/license supplied locally; no global installation or mathematical edits.

Editorial note. List-coloring and conflict-free correspondence propositions are one combined target. The first author proposition is unlabelled; its original line477–479 locator is used without inventing an author label. Auxiliary structural results and direct algorithmic/formulation consequences remain complete context without extra counts.

Editorial note. Original declaration2188–2190 states ChatGPT/Claude language/readability assistance; retained exactly without AI-free or core-discovery inference. Authors retain responsibility for mathematics; no independent proof review, certification, added hypotheses or proof repair.

Editorial note. Actual T1-font missing glyphs only: original prose 1514 double-acute o and en dash, prose1642 en dash, and original bibliography371 em dash use equivalent TeX accent/dash encodings in source0 and references-bibtex.bib. Every source line position and all mathematical content preserved. Original main.tex/bibliography.bib and failed r001 retained.

Allowed standard prerequisites (editorial).

- Partial proper colorings, list colorings, hypergraph conflict-free colorings and graph partitions. Complete native double-partition constraints C1/C2, conflict-graph incidence encoding and graph/list/hypergraph conventions retained. Original prior framework citation LucciEtAl2025 DOI10.1016/j.procs.2025.10.319 remains explicit; targeted prior-title/DOI metadata also searched with zero hits.. Both original bidirectional proof constructions and all needed definitions/figures accompany the combined target; no independently reconstructed reduction.

1. Introduction

The *Double-Partition Coloring Problem* (DPCP) was recently introduced by Lucci et al. (2025) as a unifying framework for several graph coloring problems. Given a graph $G = (V, E)$ and two partitions $\mathcal{P}$ and $\mathcal{Q}$ of V, the objective is to find a partial proper coloring such that exactly one vertex in each part of $\mathcal{P}$ is colored, and all colored vertices within each part of $\mathcal{Q}$ receive the same color. The goal is to minimize the total number of colors used.

Several well-known generalizations of the classical Vertex Coloring Problem (VCP) are polynomial-time reducible to the DPCP, including the Partition Coloring Problem (PCP) (Li & Simha, 2000), the

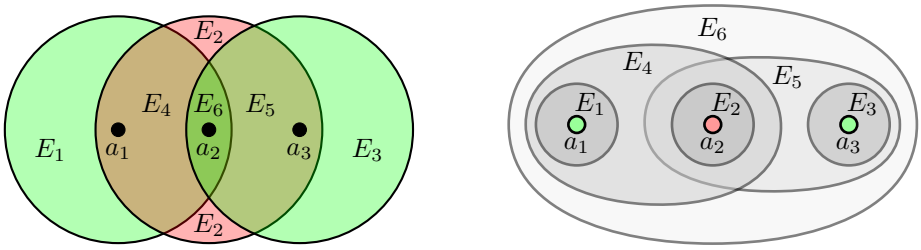


Figure 1: An example of frequency assignment for three antennas with circular coverage areas, modeled as a CFCP.

List Coloring Problem (LCP) (Lucci et al., 2024), and the Conflict-free Coloring Problem on hypergraphs (CFCP) (Even et al., 2003). All these problems arise in a wide range of applications related to resource allocation in areas such as scheduling, compilation, and telecommunications (Jensen & Toft, 2011). Exact algorithms for solving general instances have been developed, where *branch-and-price* (B&P) schemes represent the state of the art. See Mehrotra & Trick (1996); Malaguti et al. (2011) for VCP, Furini et al. (2018) for PCP, and Lucci et al. (2024) for LCP. In contrast, to the best of our knowledge, prior to Lucci et al. (2025) the CFCP had neither been studied through integer linear programming (ILP) nor provided with exact algorithms for general instances.

In the CFCP, the objective is to color a subset of the vertices of a hypergraph, while ensuring that each hyperedge contains at least one vertex whose color is not assigned to any other vertex within that hyperedge. This problem arises from an application in frequency assignment in telecommunications, where frequency channels from a finite spectrum must be assigned to antennas so that they can operate without interference. A common modeling approach represents antennas as vertices of a graph and uses edges to indicate that the corresponding antennas have intersecting coverage areas. If frequencies are represented by colors, the task becomes to color the vertices so that adjacent vertices receive different colors, that is, a VCP instance. However, this model may be overly restrictive, since an intersection of coverage areas does not necessarily imply interference, which may lead to an unnecessarily large number of frequencies. For instance, suppose that the intersection of the coverage areas of two antennas is entirely contained within the coverage area of a third antenna. Clients located in that region may connect to the latter antenna. In this case, the first two antennas may share the same frequency, provided that it differs from the frequency assigned to the third antenna. More generally, to avoid interference it suffices that a user located at any point within the network coverage area can access at least one antenna transmitting on a unique frequency. This requirement naturally leads to the CFCP.

Figure 1 illustrates the example proposed in Smorodinsky (2013). Antennas a_1, a_2, and a_3 have mutually intersecting coverage areas, as shown on the left, which under the standard VCP-based approach requires assigning a distinct frequency to each antenna. Observe, however, that the coverage area of the network can be partitioned into six regions, each characterized by the set of antennas that can be reached from that region: $E_1 = \{a_1\}$, $E_2 = \{a_2\}$, $E_3 = \{a_3\}$, $E_4 = \{a_1, a_2\}$, $E_5 = \{a_2, a_3\}$, $E_6 = \{a_1, a_2, a_3\}$. Consider the hypergraph, depicted on the right, whose vertices correspond to the three antennas and whose hyperedges are given by the sets above. If one frequency is assigned to a_1 and a_3 and another to a_2, then each hyperedge contains a vertex whose frequency is unique within that hyperedge. For example, a client located in the region corresponding to hyperedge E_6 can communicate through antenna a_2, whose frequency differs from those assigned to a_1 and a_3.

The CFCP was originally studied on hypergraphs associated with various geometric settings Har-Peled & Smorodinsky (2003); Chen et al. (2009); Pach & Tardos (2010). Later, Pach & Tardos (2009) studied the conflict-free coloring problem on general hypergraphs and on hypergraphs induced by (open or closed) neighborhoods of vertices of a graph. The authors refer to the latter problem as the *conflict-free coloring of a graph*. More recent contributions have focused on conflict-free coloring of particular graph classes such as claw-free graphs, interval graphs (Bhyravarapu et al. (2022)), planar graphs (Fabrici et al. (2023)), graphs with bounded clique width, and intersection graphs (Bhyravarapu et al. (2024)). The extension of CFCP to list conflict-free coloring has also been addressed (Gupta & Mathew (2026)).

The contributions of this work are twofold. First, we advance the structural study of the DPCP by deriving new results that enable the detection of infeasible instances and the reduction of instance size. Moreover, we provide a formal proof of the polynomial-time reductions from CFCP and LCP to DPCP, which was previously stated without proof in Lucci et al. (2025). These results establish new connections between coloring problems in graphs and hypergraphs. Second, we strengthen the B&P algorithm previously proposed for the DPCP, contributing to the development of practical exact methods for solving the CFCP. The improved algorithm integrates the new theoretical insights to preprocess instances, introduces specifically designed heuristics for the DPCP, and incorporates new methods for solving the pricing problem. Moreover, general DPCP instances are computationally evaluated for the first time using this enhanced algorithm. In particular, this evaluation includes a diverse set of CFCP instances.

The remainder of this paper is organized as follows. In the next section, required notation is introduced and the coloring problems mentioned above are formally defined. Their relationship with the DPCP is studied in Section 3. Section 4 presents fundamental structural properties of the DPCP, including a characterization of feasible instances. Heuristic approaches for solving the DPCP are presented in Section 5. The set covering integer linear programming formulation proposed in Lucci et al. (2025) is revisited in Section 6, where the main components of a B&P algorithm are described. Section 7 describes the experimental setup and the preliminary experiments used to configure the B&P algorithm. Results of computational experiments conducted both on general DPCP instances and on instances stemming from the CFCP are discussed in Section 8. Finally, Section 9 presents concluding remarks and directions for future research.

2. Notation and definitions

For $n \in \mathbb{N}$, in the following we denote by $[n]$ the set $\{z \in \mathbb{N} : z \leq n\}$.

Given a graph $G = (V, E)$, a set $K \subseteq V$ of mutually adjacent vertices of G is called a *clique* in G, and a set $S \subseteq V$ of mutually non adjacent vertices is known as a *stable set* or *independent set* in G. The *stability number* or *independence number* $\alpha(G)$ is the largest cardinality of a stable set in G. For a vertex $v \in V$, $N(v)$ denotes the set of its neighbors, $N[v] = N(v) \cup \{v\}$, and $G - v$ is the graph obtained by removing v. For a subset $U \subseteq V$, $G[U]$ denotes the subgraph of G induced by U.

A *hypergraph* $\mathcal{H}$ is a pair $(\mathcal{V}, \mathcal{E})$ where $\mathcal{V}$ is a finite set of elements called vertices and $\mathcal{E}$ is a family of non-empty subsets of $\mathcal{V}$ called hyperedges. Graphs are the special class of hypergraphs where all hyperedges have cardinality two, in which case they are simply referred to as edges.

A *coloring* of a hypergraph (or a graph) $(\mathcal{V}, \mathcal{E})$ is an assignment $c : \mathcal{V}' \to C$ of a subset $\mathcal{V}' \subseteq \mathcal{V}$ of its vertices to a set C of colors. The vertices in $\mathcal{V}'$ are the *colored* vertices. The coloring is *total* if $\mathcal{V}' = \mathcal{V}$. Coloring problems on graphs and hypergraphs have been extensively studied in the literature (see, e.g. Jensen & Toft (2011); Malaguti & Toth (2010) for two prominent survey works). We introduce the coloring problems considered throughout this work.

The classical *Vertex Coloring Problem* asks for the minimum number of colors required to define a total *proper* coloring on a graph G, i.e., a total coloring such that no two adjacent vertices receive the same color. This minimum number is called the *chromatic number* $\chi(G)$ of G.

In the *List Coloring Problem*, a list $L_v \subseteq C$ of admissible colors is given for each $v \in V$. The task is to find a total proper coloring that satisfies $c(v) \in L_v$ for every $v \in V$ and minimizes the number of required colors (Lucci et al., 2024). The feasible solutions of LCP are termed as *list colorings*. Note that LCP is a generalization of VCP. Indeed, if $L_v = C$ holds for all $v \in V$, then list colorings are exactly the total proper colorings of G.

A coloring $c : \mathcal{V}' \to C$ of a hypergraph $\mathcal{H} = (\mathcal{V}, \mathcal{E})$, with $\mathcal{V}' \subseteq \mathcal{V}$, is called *conflict-free* if each hyperedge $E \in \mathcal{E}$ contains a vertex of *unique color*, i.e., a vertex $v \in E \cap \mathcal{V}'$ satisfying $|\{w \in E : c(w) = c(v)\}| = 1$. The *Conflict-Free Coloring Problem* is the task to determine a conflict-free coloring of $\mathcal{H}$ minimizing the number of required colors (Even et al., 2003). Its optimum value is known as the *conflict-free chromatic number* of $\mathcal{H}$ and is denoted by $\chi_{CF}(\mathcal{H})$. The total conflict-free chromatic number $\chi^T_{CF}(\mathcal{H})$ is the minimum number of colors required to define a total conflict-free coloring of $\mathcal{H}$. Observe that $\chi_{CF}(\mathcal{H}) \leq \chi^T_{CF}(\mathcal{H})$ holds trivially. Moreover, $\chi^T_{CF}(\mathcal{H}) \leq \chi_{CF}(\mathcal{H}) + 1$, as any conflict-free coloring of $\mathcal{H}$ can be extended to a total conflict-free coloring by assigning a new color to each uncolored vertex of $\mathcal{H}$. Total CFCP is a generalization of VCP, as it is straightforward to verify that both problems coincide on graphs. Indeed, any total proper coloring of a graph G is a total conflict-free coloring, as each edge of G contains at least one (actually two) uniquely colored vertices. Conversely, if $uv \in E$, then any total conflict-free coloring of G assigns two different (unique) colors to the vertices u and v. Hence, $\chi^T_{CF}(G) = \chi(G)$ holds for any graph G.

Given a partition $\mathcal{P} = \{P_i : i \in [k]\}$ of the set of vertices of a graph $G = (V, E)$, the *Partition Coloring Problem* asks for a proper coloring of G such that exactly one vertex of each partition class is colored, while the number of required colors is minimized (Li & Simha, 2000). Observe that VCP is the special case of PCP obtained when $\mathcal{P} = \{\{v\} : v \in V\}$.

3. The Double-Partition Coloring Problem

Given a graph $G = (V, E)$, and two partitions $\mathcal{P}$ and $\mathcal{Q}$ of V, a *double-partition coloring* of G is a proper coloring of G that satisfies the following two constraints:

(C1) For each $P \in \mathcal{P}$, exactly one vertex of P is colored.

(C2) For each $Q \in \mathcal{Q}$, all colored vertices in Q are assigned the same color.

The *Double-Partition Coloring Problem* (DPCP) is the optimization problem of finding the minimum number of colors required to define a double-partition coloring on G, referred to as its *double-partition chromatic number* $\chi_{DP}(G, \mathcal{P}, \mathcal{Q})$. Formally,

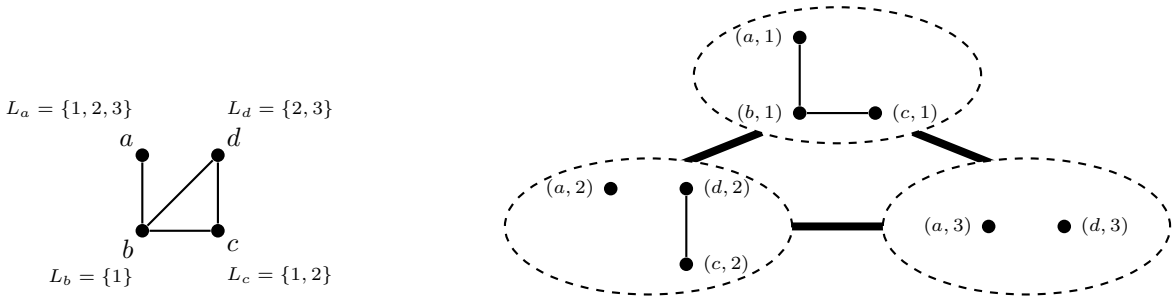


Figure 2: Example of the construction of $\tilde{G}$ (right) from graph G (left). Each subgraph on the right contains the vertices of a part Q_k, $k \in C$. Each thick segment represents the complete join between the corresponding vertex sets.

Double-Partition Coloring Problem (DPCP)
INSTANCE. A graph $G = (V, E)$ and two partitions $\mathcal{P}$ and $\mathcal{Q}$ of V.
OBJECTIVE. To obtain $\chi_{DP}(G, \mathcal{P}, \mathcal{Q})$.

In the remainder of this paper $(G, \mathcal{P}, \mathcal{Q})$ denotes an instance of DPCP with $\mathcal{P} = \{P_i : i \in [n]\}$ and $\mathcal{Q} = \{Q_j : j \in [m]\}$. Given a vertex v, we denote by $i(v) \in [n]$ (resp. $j(v) \in [m]$) the unique index such that $v \in P_{i(v)}$ (resp. $v \in Q_{j(v)}$).

Remark 3.1. *For each $P \in \mathcal{P}$, adjacencies among vertices in P impose no restriction on the coloring, since exactly one vertex in P is assigned a color. This observation generalizes Proposition 2 in Li & Simha (2000) for the PCP. Therefore, throughout this paper, each set P is assumed to be either a clique or an independent set, whichever is more convenient for the arguments.*

No reductions are known between LCP, PCP, and CFCP. However, we will show that each of these problems can be reduced to specific instances of DPCP.

Clearly, PCP is obtained from DPCP when $\mathcal{Q} = \{\{v\} : v \in V\}$. In fact, the same holds for any partition $\mathcal{Q}$ satisfying that each part $Q \in \mathcal{Q}$ is contained in some part $P \in \mathcal{P}$.

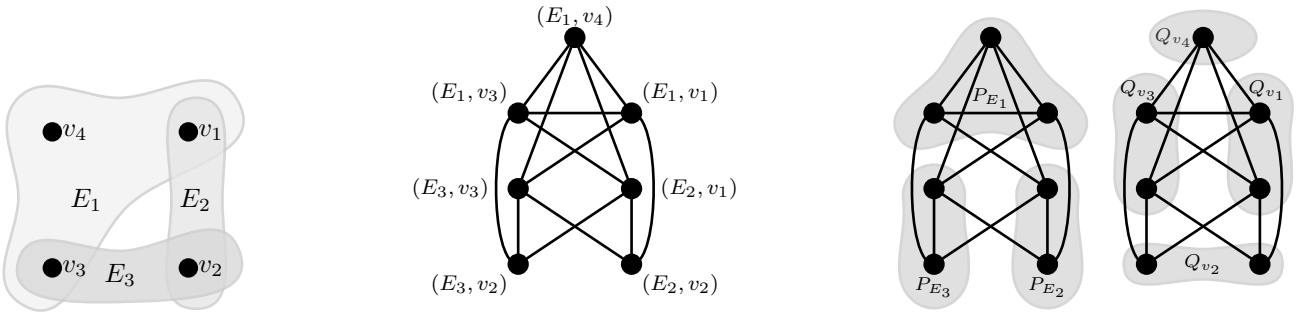
Moreover, consider an instance of LCP defined by a graph $G = (V, E)$, a color set C, and, for each $v \in V$, a set $L_v \subseteq C$ of admissible colors. Define $\tilde{G} = (\tilde{V}, \tilde{E})$ as the graph with vertex set $\tilde{V} = \{(v, k) : v \in V,\ k \in L_v\}$, and edge set $\tilde{E}$ such that $(v_1, k_1)(v_2, k_2) \in \tilde{E}$ if and only if $v_1 v_2 \in E$ or $k_1 \neq k_2$. Let

$$\mathcal{P} = \{P_v : v \in V\},\ \text{with } P_v = \{(v', k) \in \tilde{V} : v' = v\},\ \text{and}$$
$$\mathcal{Q} = \{Q_k : k \in C\},\ \text{with } Q_k = \{(v, k') \in \tilde{V} : k' = k\}$$

be two partitions of $\tilde{V}$. Figure 2 illustrates this construction. Then, the following result can be proved:

Proposition 3.1. *Each list coloring of G induces a double-partition coloring of $(\tilde{G}, \mathcal{P}, \mathcal{Q})$ with the same number of colors, and conversely.*

Proof. Let $c : V \longrightarrow C$ be a list coloring of G and $\tilde{V}' = \{(v, c(v)) : v \in V\} \subseteq \tilde{V}$. Consider the color assignment $\tilde{c} : \tilde{V}' \to C$ defined by $\tilde{c}(v, c(v)) = c(v)$. Clearly, $\tilde{c}$ verifies constraints (C1) and (C2), and uses as many colors as c. To prove that $\tilde{c}$ is a double-partition coloring of $\tilde{G}$, it remains to show that $\tilde{c}$ is a proper coloring of $\tilde{G}$. Indeed, let (v_1, k_1) and (v_2, k_2) be two adjacent colored vertices of $\tilde{G}$. By the definition of $\tilde{E}$, either $v_1 v_2 \in E$ or $k_1 \neq k_2$. Moreover, observe that $\tilde{c}(v_1, k_1) = k_1 = c(v_1)$ and $\tilde{c}(v_2, k_2) = k_2 = c(v_2)$. Thus, if $v_1 v_2 \in E$, then $c(v_1) \neq c(v_2)$ as c is a proper coloring of G. Otherwise, $k_1 \neq k_2$ must hold. In both cases, it follows that $\tilde{c}(v_1, k_1) \neq \tilde{c}(v_2, k_2)$.



(a) Hypergraph $\mathcal{H}$. (b) Conflict graph $G(\mathcal{H})$. (c) Partitions $\mathcal{P}$ and $\mathcal{Q}$ of $V(G(\mathcal{H}))$.

Figure 3: Example of the construction of the conflict graph $G(\mathcal{H})$ for the hypergraph $\mathcal{H}$.

For the converse, let $\tilde{c}$ be a double-partition coloring for $\tilde{G}$. Due to constraint (C1), for each $v \in V$ there is a unique $k_v \in L_v$ such that the vertex $(v, k_v) \in P_v$ is colored. Observe that two vertices (v_1, k_{v_1}) and (v_2, k_{v_2}) have the same color if and only if $k_{v_1} = k_{v_2}$. Indeed, if $k_{v_1} = k_{v_2}$ then $(v_1, k_{v_1}), (v_2, k_{v_2}) \in Q_{k_{v_1}} = Q_{k_{v_2}}$ and constraint (C2) requires both vertices to have the same color. Otherwise, if $k_{v_1} \neq k_{v_2}$, the vertices are adjacent and must have different colors, as $\tilde{c}$ is proper. As a consequence, the number of colors required by $\tilde{c}$ is equal to $|\{k_v : v \in V\}|$.

Now consider the color assignment $c : V \longrightarrow C$ defined by $c(v) = k_v \in L_v$, which clearly requires the same number of colors as $\tilde{c}$. To prove that c is a proper coloring of G, let v_1, v_2 be two adjacent vertices from V. By the definition of $\tilde{E}$, (v_1, k_{v_1}) and (v_2, k_{v_2}) are adjacent in $\tilde{G}$. Hence, $\tilde{c}(v_1, k_{v_1}) \neq \tilde{c}(v_2, k_{v_2})$, as $\tilde{c}$ is a proper coloring of $\tilde{G}$. Moreover, $k_{v_1} \neq k_{v_2}$ follows from the observation above and thus $c(v_1) \neq c(v_2)$. □

To clarify the relationship between DPCP and CFCP, we first introduce the notion of the *conflict graph of a hypergraph*.

Definition 3.1. *Given a hypergraph $\mathcal{H} = (\mathcal{V}, \mathcal{E})$ its conflict graph $G(\mathcal{H})$ is the graph whose vertex set is*

$$V(G(\mathcal{H})) = \{(E, v) : E \in \mathcal{E}, v \in E\},$$

and whose edge set is defined as follows:

$$(E_1, v_1)(E_2, v_2) \in E(G(\mathcal{H})) \Leftrightarrow v_1 \neq v_2 \text{ and } (v_1 \in E_2 \text{ or } v_2 \in E_1)$$

The idea underlying the definition above is that assigning a color k to a vertex (E, v) of $G(\mathcal{H})$ encodes two decisions in the hypergraph $\mathcal{H}$: assigning color k to vertex v in hyperedge E, and forbidding any other vertex of E from receiving the same color k. Consequently, two vertices (E_1, v_1) and (E_2, v_2) of $G(\mathcal{H})$ with $v_1 \neq v_2$ cannot be assigned the same color whenever $v_2 \in E_1$ or $v_1 \in E_2$. Therefore, such vertices must be adjacent in $G(\mathcal{H})$. Moreover, consider the two partitions of $V(G(\mathcal{H}))$ defined by:

$$\mathcal{P} = \{P_E : E \in \mathcal{E}\}, \text{ with } P_E = \{(E, v') \in V(G(\mathcal{H})) : v' \in E\}, \text{ and}$$
$$\mathcal{Q} = \{Q_v : v \in V\}, \text{ with } Q_v = \{(E', v) \in V(G(\mathcal{H})) : v \in E'\}.$$

By Definition 3.1, P_E is a clique for each $E \in \mathcal{E}$, since for any two distinct vertices $(E, v_1), (E, v_2) \in P_E$, we have $v_1 \in E$ and $v_2 \in E$. An example of this construction is illustrated in Figure 3, adapted from an example presented in Lucci et al. (2025). Then, the following result can be proved.

Proposition 3.2. *Each conflict-free coloring of $\mathcal{H}$ induces a double-partition coloring of $(G(\mathcal{H}), \mathcal{P}, \mathcal{Q})$ with the same number of colors, and conversely. Therefore,*

$$\chi_{CF}(\mathcal{H}) = \chi_{DP}(G(\mathcal{H}), \mathcal{P}, \mathcal{Q}).$$

Proof. Let $c : \mathcal{V}' \to C$, with $\mathcal{V}' \subseteq \mathcal{V}$, be a conflict-free coloring of $\mathcal{H}$. For each $E \in \mathcal{E}$, let $v_E \in E$ be a uniquely colored vertex in E with respect to c. Define $V' = \{(E, v_E) : E \in \mathcal{E}\} \subseteq V(G(\mathcal{H}))$, and let $\tilde{c} : V' \to C$ be the coloring of $G(\mathcal{H})$ defined by $\tilde{c}(E, v_E) = c(v_E)$. Clearly, $\tilde{c}$ uses no more colors than c. Moreover, by construction, $\tilde{c}$ satisfies conditions (C1) and (C2).

We now show that $\tilde{c}$ is a proper coloring of $G(\mathcal{H})$. Let (E_1, v_{E_1}) and (E_2, v_{E_2}) be adjacent vertices in V'. By Definition 3.1, we have $v_{E_1} \neq v_{E_2}$ and either $v_{E_2} \in E_1$ or $v_{E_1} \in E_2$. Assume, without loss of generality, that $v_{E_2} \in E_1$. Since v_{E_1} is the unique vertex of E_1 colored with $c(v_{E_1})$, it follows that $c(v_{E_1}) \neq c(v_{E_2})$, and therefore $\tilde{c}(E_1, v_{E_1}) \neq \tilde{c}(E_2, v_{E_2})$. Thus, $\tilde{c}$ is proper.

For the converse direction, let $\tilde{c} : \tilde{V}' \to C$, with $\tilde{V}' \subseteq V(G(\mathcal{H}))$, be a double-partition coloring of $(G(\mathcal{H}), \mathcal{P}, \mathcal{Q})$. For each $v \in \mathcal{V}$, define $Q'_v = Q_v \cap \tilde{V}'$, and let $\mathcal{W} = \{v \in \mathcal{V} : Q'_v \neq \emptyset\}$. By condition (C2), all vertices in Q'_v receive the same color. Hence the mapping $c : \mathcal{W} \to C$ defined by $c(v) = \tilde{c}(E, v)$, for any $(E, v) \in Q'_v$, is a well-defined coloring of $\mathcal{H}$ and uses the same number of colors as $\tilde{c}$.

Finally, we prove that c is conflict-free. In fact, condition (C1)states that, for any $E \in \mathcal{E}$, exactly one vertex $(E, v_E) \in P_E$ is colored. Thus, v_E is colored in c. Suppose there exists another colored vertex $v' \in E$, $v' \neq v_E$. Then $(E', v') \in \tilde{V}'$ holds for some $E' \in \mathcal{E}$, $E' \neq E$. Moreover, observe that (E, v_E) and (E', v') are adjacent in $G(\mathcal{H})$. Since $\tilde{c}$ is proper, $\tilde{c}(E, v_E) \neq \tilde{c}(E', v')$, and therefore $c(v_E) \neq c(v')$. Hence, v_E has a color distinct from all other colored vertices in E, and c is a conflict-free coloring of $\mathcal{H}$. □

As a consequence of Proposition 3.2, DPCP generalizes CFCP. Moreover, this generalization is strict, since all CFCP instances are feasible, while there are infeasible instances of DPCP, as we shall see in the next section.

4. First results on particular instances of DPCP

In this section, we analyze DPCP instances with special structures that yield preprocessing rules, which either reduce the problem to smaller equivalent instances or detect infeasibility.

Recall that $(G, \mathcal{P}, \mathcal{Q})$ denotes an instance of the DPCP, with $G = (V, E)$, $\mathcal{P} = \{P_i : i \in [n]\}$, and $\mathcal{Q} = \{Q_j : j \in [m]\}$. Due to Remark 3.1, we assume that each set in $\mathcal{P}$ induces a clique in G.

Let us first introduce the following remark for instances where $n = 1$ or $m = 1$.

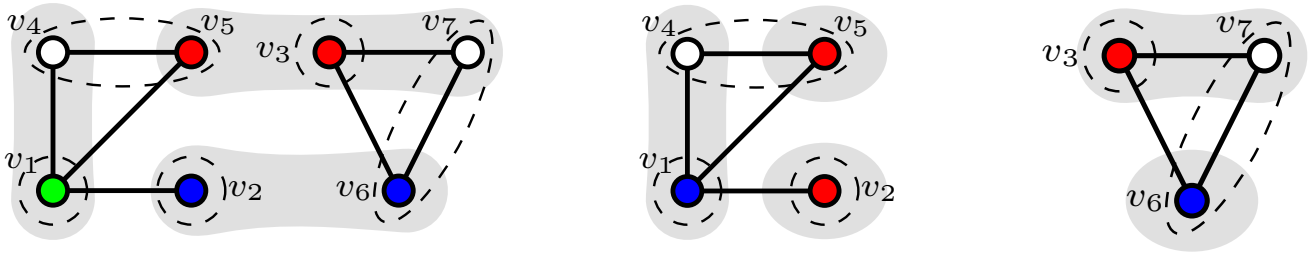
Remark 4.1. *Instances $(G, \mathcal{P}, \mathcal{Q})$ with $\mathcal{P} = \{V\}$ are always feasible and $\chi_{DP}(G, \mathcal{P}, \mathcal{Q}) = 1$. Indeed, assigning an arbitrary color to any vertex $v \in V$ yields an optimal double-partition coloring. For instances $(G, \mathcal{P}, \mathcal{Q})$ with $\mathcal{Q} = \{V\}$, since all vertices must share the same color and each $P \in \mathcal{P}$ is a clique, $(G, \mathcal{P}, \mathcal{Q})$ is feasible if and only if $\alpha(G) = n$. In this case, all colored vertices belong to the same part $Q = V$ and, by (C2), must therefore receive the same color. Hence, exactly one color is used, and $\chi_{DP}(G, \mathcal{P}, \mathcal{Q}) = 1$.*

From the previous remark, in the following we consider DPCP instances where n and m at least two.

Observe that, if $P = \{v\}$ for some part $P \in \mathcal{P}$, then v must always be colored and this leads to the following possible instance reductions:

Lemma 4.1. *Given $(G, \mathcal{P}, \mathcal{Q})$, let $P \in \mathcal{P}$ such that $P = \{u\}$ and $v \in Q_{j(u)} \setminus \{u\}$.*

- *If $v \in N(u)$ and $P_{i(v)} = \{v\}$, then $(G, \mathcal{P}, \mathcal{Q})$ is infeasible.*



(a) $(G, \mathcal{P}, \mathcal{Q})$ (b) $(G_1, \mathcal{P}^1, \mathcal{Q}^1)$ (c) $(G_2, \mathcal{P}^2, \mathcal{Q}^2)$

Figure 4: (a) An optimal double-partition coloring of $(G, \mathcal{P}, \mathcal{Q})$; (b) and (c) optimal double-partition colorings of the subinstances $(G_1, \mathcal{P}^1, \mathcal{Q}^1)$ and $(G_2, \mathcal{P}^2, \mathcal{Q}^2)$, respectively.

- *If $v \in N(u)$ and $|P_{i(v)}| \geq 2$, then solving $(G, \mathcal{P}, \mathcal{Q})$ is equivalent to solving the instance $(G-v, \tilde{\mathcal{P}}, \tilde{\mathcal{Q}})$, where $\tilde{\mathcal{P}} = \{P \setminus \{v\} : P \in \mathcal{P}\}$ and $\tilde{\mathcal{Q}} = \{Q \setminus \{v\} : Q \in \mathcal{Q}\}$.*
- *If $v \notin N(u)$ and $P_{i(v)} = \{v\}$ then solving $(G, \mathcal{P}, \mathcal{Q})$ is equivalent to solving $(\tilde{G}, \tilde{\mathcal{P}}, \tilde{\mathcal{Q}})$ where $\tilde{G}$ is the graph obtained by contracting u and v, i.e. $\tilde{G} = (V \setminus \{v\}, \tilde{E})$, with $\tilde{E} = (E \setminus \{vw : w \in N(v)\}) \cup \{uw : w \in N(v)\}$, $\tilde{\mathcal{P}} = \mathcal{P} \setminus \{\{v\}\}$ and $\tilde{\mathcal{Q}} = \{Q \setminus \{v\} : Q \in \mathcal{Q}\}$.*

Proof. Since $P = \{u\}$, node u must be colored in order to satisfy condition (C1). Moreover, since $v \in Q_{j(u)}$, if v is colored, then it must be assigned to the same color as u.

Therefore, if $v \in N(u)$, nodes u and v cannot both be colored, and hence v must remain uncolored.

If, in addition, $P_{i(v)} = \{v\}$, then v must be colored to satisfy condition (C1) for $P_{i(v)}$, which contradicts the fact that v must remain uncolored. Thus, the instance is infeasible.

If $|P_{i(v)}| \geq 2$, the fact that v must remain uncolored implies that v can be removed from the graph and from all sets in $\mathcal{P}$ and $\mathcal{Q}$. The resulting instance is equivalent to $(G, \mathcal{P}, \mathcal{Q})$, proving the second statement.

Finally, suppose that $v \notin N(u)$ and $P_{i(v)} = \{v\}$. In this case, v must be colored, and, since $v \in Q_{j(u)}$, both u and v must be assigned to the same color. Hence, u and v can be identified, yielding an equivalent instance obtained by contracting u and v as described in the third statement. □

Observe that, by iteratively applying this last result, we arrive at either an infeasible instance or an instance where, for every vertex v such that $|P_{i(v)}| = 1$, $Q_{j(v)}$ is a stable set.

Many graph coloring problems can be decomposed according to the connected components of the underlying graph or hypergraph. For example, instances of the VCP, PCP, and CFCP can be solved independently on each connected component, and the resulting subsolutions can be merged into a complete solution. However, DPCP instances do not follow this pattern when some part in $\mathcal{Q}$ intersects more than one connected component. Observe that this cannot occur for any part $P \in \mathcal{P}$, since P induces a clique and is therefore contained in a single connected component.

Example 4.1. *Figure 4(a) shows an optimal double-partition coloring with 3 colors for an instance $(G, \mathcal{P}, \mathcal{Q})$, where the parts of $\mathcal{P}$ are represented by dashed lines and those of $\mathcal{Q}$ are shaded. Note that G has two connected components, G_1 and G_2, and each part $P \in \mathcal{P}$ is fully contained in either $V(G_1)$ or $V(G_2)$. This, however, does not hold for all parts of $\mathcal{Q}$. Figure 4(b) and (c) show optimal double-partition colorings with 2 colors for the subinstances $(G_1, \mathcal{P}^1, \mathcal{Q}^1)$ and $(G_2, \mathcal{P}^2, \mathcal{Q}^2)$ induced by the connected components G_1 and G_2, respectively. Merging these colorings does not yield a feasible double-partition coloring for the original instance, since v_2 and v_6 receive different colors even though they belong to the same part of $\mathcal{Q}$.*

However, decomposition in connected components can be exploited to obtain lower bounds on the double-partition chromatic number of a graph. Indeed, the restriction of a double-partition coloring

of $(G, \mathcal{P}, \mathcal{Q})$ to the vertex set V_k of a connected component G_k, yields a double-partition coloring of $(G_k, \mathcal{P}^k, \mathcal{Q}^k)$, where $\mathcal{P}^k = \{P \cap V_k : P \in \mathcal{P},\ P \cap V_k \neq \emptyset\}$ and $\mathcal{Q}^k = \{Q \cap V_k : Q \in \mathcal{Q},\ Q \cap V_k \neq \emptyset\}$. Therefore, $\chi_{DP}(G_k, \mathcal{P}^k, \mathcal{Q}^k) \leq \chi_{DP}(G, \mathcal{P}, \mathcal{Q})$.

If each part of $\mathcal{Q}$ intersects exactly one connected component of G, then the instance admits the following decomposition.

Lemma 4.2. *Let G be a graph with ℓ connected components, each denoted by $G_k = (V_k, E_k)$ with $k \in [\ell]$, such that, for every $Q \in \mathcal{Q}$, $Q \subseteq V_k$ for some $k \in [\ell]$. For every $k \in [\ell]$, let $\mathcal{P}^k = \{P \cap V_k : P \in \mathcal{P},\ P \cap V_k \neq \emptyset\}$ and $\mathcal{Q}^k = \{Q \cap V_k : Q \in \mathcal{Q},\ Q \cap V_k \neq \emptyset\}$. Then, $(G, \mathcal{P}, \mathcal{Q})$ is feasible if and only if $(G_k, \mathcal{P}^k, \mathcal{Q}^k)$ is feasible for every $k \in [\ell]$. Moreover, if $(G, \mathcal{P}, \mathcal{Q})$ is feasible, then $\chi_{DP}(G, \mathcal{P}, \mathcal{Q}) = \max\{\chi_{DP}(G_k, \mathcal{P}^k, \mathcal{Q}^k) : k \in [\ell]\}$.*

The proof follows immediately from the fact that in this case a feasible double-partitioning coloring for $(G, \mathcal{P}, \mathcal{Q})$ can be obtained by merging the colorings of the components.

If G has a connected component consisting of an isolated vertex v, then removing v preserves feasibility and does not change the optimal value, regardless of whether $Q_{j(v)}$ intersects different connected components.

Lemma 4.3. *Let G be a graph with an isolated vertex v and $|V| \geq 2$. Define $G' = G \setminus \{v\}$, $\mathcal{P}' = \mathcal{P} \setminus \{\{v\}\}$, and $\mathcal{Q}' = \{Q \setminus \{v\} : Q \in \mathcal{Q}\}$. Then, $(G, \mathcal{P}, \mathcal{Q})$ is feasible if and only if $(G', \mathcal{P}', \mathcal{Q}')$ is feasible. Moreover, if $(G', \mathcal{P}', \mathcal{Q}')$ is feasible, $\chi_{DP}(G, \mathcal{P}, \mathcal{Q}) = \chi_{DP}(G', \mathcal{P}', \mathcal{Q}')$.*

The proof is straightforward, since any optimal double-partition coloring of $(G', \mathcal{P}', \mathcal{Q}')$ can be extended to an optimal coloring of $(G, \mathcal{P}, \mathcal{Q})$ by assigning to v the color used in $Q_{j(v)}$, if any vertex in $Q_{j(v)}$ is colored, or any color used in the solution, otherwise.

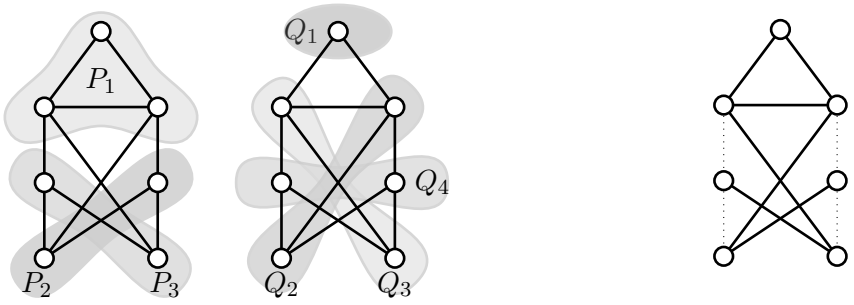
Finally we analyze feasibility of DPCP instances. For the VCP, PCP, and CFCP, feasibility of any instance is guaranteed, provided that the number of colors is sufficiently large; however, this does not hold for the LCP. In fact, deciding feasibility for the LCP is NP-complete (Golovach et al., 2017), which in turn implies the NP-completeness of the feasibility problem for the DPCP. The smallest infeasible instance of the DPCP is given by the graph $G = (V, E)$ with $V = \{u, v\}$, $E = \{uv\}$, together with the partitions $\mathcal{P} = \{\{u\}, \{v\}\}$ and $\mathcal{Q} = \{\{u, v\}\}$.

We define a *set of representatives of* $(G, \mathcal{P}, \mathcal{Q})$ as any subset $W \subseteq V$ such that $|W \cap P| = 1$ for each $P \in \mathcal{P}$. Note that any double-partition coloring of $(G, \mathcal{P}, \mathcal{Q})$ induces a set W of representatives (namely, he colored nodes) and a proper coloring c of $G[W]$.

Unlike the PCP, given an arbitrary set W of representatives, a proper coloring of the graph $G[W]$ cannot necessarily be extended to a double-partition coloring of $(G, \mathcal{P}, \mathcal{Q})$. This is because the DPCP additionally requires all vertices belonging to the same $Q \in \mathcal{Q}$ to be assigned the same color. Consequently, we must ensure that the intersection of W with each $Q \in \mathcal{Q}$ is a stable set in G. A set of representatives satisfying the latter condition will in the following be called a *feasible set of representatives.*

Remark 4.2. *The set of colored vertices of any double-partition coloring of $(G, \mathcal{P}, \mathcal{Q})$ is a feasible set of representatives of $(G, \mathcal{P}, \mathcal{Q})$. Conversely, if W is a feasible set of representatives, assigning one color for each set $W \cap Q$, $Q \in \mathcal{Q}$, yields a double-partition coloring of $(G, \mathcal{P}, \mathcal{Q})$. Therefore, $(G, \mathcal{P}, \mathcal{Q})$ is feasible if and only if there exists a feasible set of representatives.*

To compute $\chi_{DP}(G, \mathcal{P}, \mathcal{Q})$, given a subset of vertices W, we define the graph $G[W]/\mathcal{Q}$ with a vertex w_j for each $j \in [m]$ such that $W \cap Q_j \neq \emptyset$. Two vertices w_{j_1} and w_{j_2} are adjacent in $G[W]/\mathcal{Q}$ if and only if there exist $v_1 \in W \cap Q_{j_1}$ and $v_2 \in W \cap Q_{j_2}$ such that $v_1 v_2 \in E$.



(a) $(G, \mathcal{P}, \mathcal{Q})$ (b) $\tilde{G} = (V, \tilde{E})$

Figure 5: Instance $(G, \mathcal{P}, \mathcal{Q})$ is feasible if and only if $\tilde{G}$ contains a stable set of size $|\mathcal{P}| = 3$.

Proposition 4.1. *For any feasible instance $(G, \mathcal{P}, \mathcal{Q})$ of DPCP,*

$$\chi_{DP}(G, \mathcal{P}, \mathcal{Q}) = \min\{\chi(G[W]/\mathcal{Q}) : W \text{ is a feasible set of representatives}\}.$$

Proof. It is enough to prove that the minimum number of colors in a double-partition coloring defined on a feasible set of representatives W is $\chi(G[W]/\mathcal{Q})$.

A double-partition coloring of $(G, \mathcal{P}, \mathcal{Q})$ defined on W corresponds to a total proper coloring of $G[W]/\mathcal{Q}$ in which each vertex $w_j \in V(G[W]/\mathcal{Q})$ is assigned the unique color used in Q_j. Conversely, since $W \cap Q$ is a stable set of G for every $Q \in \mathcal{Q}$, a total proper coloring of $G[W]/\mathcal{Q}$ corresponds to a double-partition coloring of $(G, \mathcal{P}, \mathcal{Q})$ in which each $v \in W$ is assigned the color of $w_{j(v)}$. Both constructions preserve the number of colors used. □

Remark 4.2 shows that the feasibility of an instance does not depend on the edges between distinct parts of $\mathcal{Q}$. Therefore, by removing these edges while retaining those that make each part of $\mathcal{P}$ a clique, we can reduce the feasibility problem to the Maximum Stable Set Problem. Formally,

Proposition 4.2. *Given $(G, \mathcal{P}, \mathcal{Q})$ with $G = (V, E)$, let $\tilde{G} = (V, \tilde{E})$, with $\tilde{E} = \{uv \in E : i(u) = i(v) \vee j(u) = j(v)\}$. Then, $(G, \mathcal{P}, \mathcal{Q})$ is feasible if and only if $\alpha(\tilde{G}) = n$.*

Proof. It is clear that $\alpha(\tilde{G}) \leq n$, as any set of more than n vertices contains at least two vertices belonging to the same $P \in \mathcal{P}$, which are adjacent in $\tilde{G}$.

Let $(G, \mathcal{P}, \mathcal{Q})$ be a feasible instance of DPCP. By the previous lemma, there exists a set of representatives W such that, for all $Q \in \mathcal{Q}$, $W \cap Q$ is a stable set of G. Then, for any $u, v \in W$ we have $i(u) \neq i(v)$ and $j(u) \neq j(v)$, i.e., $uv \notin \tilde{E}$. Therefore, W is a stable set of $\tilde{G}$ and $|W| = n$. We can conclude that $\alpha(\tilde{G}) = n$.

Conversely, assume $\alpha(\tilde{G}) = n$ and let $S \subseteq V$ be a stable set of maximum cardinality in $\tilde{G}$. Thus, we have $|S \cap P| = 1$ for all $P \in \mathcal{P}$. Furthermore, consider two nodes $u, v \in S \cap Q$, for some $Q \in \mathcal{Q}$. Since S is a stable set in $\tilde{G}$, $uv \notin \tilde{E}$ and it follows that $uv \notin E$. Therefore, $S \cap Q$ is a stable set of G for each $Q \in \mathcal{Q}$. □

An example of the construction described in Proposition 4.2 is illustrated in Figure 5.

5. Heuristic approaches for DPCP

This section describes heuristic approaches for DPCP based on techniques originally developed for PCP. A key difference between the two problems is that DPCP may admit infeasible instances, which makes feasibility a central concern in the design of these heuristics.

For PCP, Li & Simha (2000) propose two strategies depending on whether vertex selection and color assignment are carried out sequentially or simultaneously. *Two-step* heuristics first select a vertex from each part of the first partition and then color the subgraph induced by these vertices. In contrast, in *one-step* heuristics, a valid color is immediately assigned to a selected vertex before the next vertex is chosen. The criteria according to which vertices and colors are selected are the main design choices and give rise to different, yet related, heuristics. Among these, the color-degree criteria, inspired by the DSatur heuristic for VCP (Brélaz, 1979), exhibited the best performance, particularly within the one-step approach, referred to as *onestepCD*.

In what follows, we consider a DPCP instance $(G, \mathcal{P}, \mathcal{Q})$, with $G = (V, E)$, $\mathcal{P} = \{P_i : i \in [n]\}$, and $\mathcal{Q} = \{Q_j : j \in [m]\}$. For convenience, this section assumes that each $P_i \in \mathcal{P}$ is a stable set of G, so that the degree of v is not influenced by the size of $P_{i(v)}$. Any tie arising in the application of the heuristics is ultimately resolved according to an index-based criterion.

5.1. Two-step heuristics

For DPCP, the vertex-selection phase aims to construct, whenever possible, a feasible set of representatives W. Algorithm 1 presents a greedy heuristic for constructing the set W iteratively. Let I denote the set of indices corresponding to the uncolored parts of $\mathcal{P}$. At each iteration, an uncolored part P_i with $i \in I$ and a vertex $v \in P_i$ are chosen according to a given criterion. Before proceeding to the next iteration, all vertices in $P_i \setminus \{v\}$ and all neighbors of v in $Q_{j(v)}$ are removed from the instance (from the graph and the partitions), thereby avoiding superfluous vertices and ensuring that $W \cap Q_{j(v)}$ remains a stable set, respectively. After these removals, it may happen that some part of $\mathcal{P}$ becomes empty, in which case the heuristic fails.

Algorithm 1: greedy heuristic for the vertex-selection phase

```
   Data: G = (V, E) and partitions P,Q of V
1  W ← ∅; I ← [n];
2  while I ≠ ∅ do
3  |  select i ∈ I and v ∈ P_i with some criterion;
4  |  W ← W ∪ {v}; I ← I \ {i};
5  |  remove vertices in P_i \ {v} and N(v) ∩ Q_{j(v)} from the instance;
6  |  if P_k = ∅ for some k ∈ I then
7  |  |  fail;
8  return W;
```

Regardless of the vertex-selection criterion, Algorithm 1 is guaranteed to terminate successfully whenever every $Q_j \in \mathcal{Q}$ induces a stable set in G. This immediately yields the following result.

Remark 5.1. *For instances of DPCP arising from VCP, PCP and CFCP, Algorithm 1 always succeeds.*

However, for general instances of DPCP, as well as for instances arising from LCP, Algorithm 1 may fail to produce a feasible solution. To increase the likelihood of obtaining a feasible solution, in line 3 we select an uncolored set $P_i \in \mathcal{P}$ of minimum cardinality. We then consider three criteria for choosing the vertex v from P_i. All criteria are evaluated on the current graph at the corresponding iteration. Ties are broken by selecting the vertex v with the fewest neighbors in $Q_{j(v)}$.

DEG. Select a vertex of minimum degree (used by Li & Simha (2000) for PCP).

QDeg. Select a vertex v whose neighborhood intersects the fewest parts of $\mathcal{Q}$. An adaptation of Deg that takes into account the future contraction of the parts of $\mathcal{Q}$ before coloring.

SQDeg. For each candidate vertex v, compute its degree in the graph obtained by contracting the parts of $\mathcal{Q}$ in $G[W \cup \{v\}]$, and select the vertex with minimum degree. Here, W denotes the set of already selected vertices. Unlike Deg and QDeg, this criterion ignores adjacencies involving non-selected vertices, many of which may never be selected.

For the coloring phase, Proposition 4.1 implies that the problem can be reduced to solve VCP on the graph $G[W]/\mathcal{Q}$. Although any VCP algorithm could be used for this task, an optimal coloring of $G[W]/\mathcal{Q}$ does not necessarily yield an optimal double-partition coloring of the original DPCP instance, since the set W is fixed. Therefore, we employ the classical DSatur heuristic of Brélaz (1979) to color $G[W]/\mathcal{Q}$.

Motivated by the good performance of the two-step heuristics, which will be discussed in Section 7, we propose a semi-greedy variant of Algorithm 1 by introducing randomness through a restricted candidate list (RCL) (Feo & Resende, 1995). Vertices whose value under the selection criterion lies within $\alpha\%$ of the best are included in the RCL, and one of them is chosen at random and added to W. The modified heuristic is run a fixed number of times, each time followed by the coloring phase. The best solution found, i.e., the one using the fewest colors, is returned.

5.2. One-step heuristics

The overall structure of the one-step greedy heuristics for DPCP is fairly similar to Algorithm 1, with two additional features: a color is immediately assigned once a vertex is selected, and additional vertices that conflict with this assignment are removed. The resulting coloring is returned at the end. Further details on the required modifications are specified below.

After a vertex v is selected in line 3, its color must be carefully chosen to maintain feasibility, unlike in PCP, where any color not used by its neighbors can be assigned. This implies that, if v is not the first colored vertex in $Q_{j(v)}$, it must receive the color already used in $Q_{j(v)}$. Otherwise, a color not used by the neighbors of v must be selected. However, this choice of color is not arbitrary, since each color invalidates a specific set of vertices. Specifically, assigning a color c to v removes all vertices in $Q_{j(v)}$ that are adjacent to a vertex colored with c, as well as any neighbor u of v such that $j_u \neq j_v$ and $Q_{j(u)}$ already contains a vertex colored with c. These vertices are removed after line 5, following the removals triggered by the selection of v.

Our vertex-selection criterion is directly inspired by *onestepCD* for PCP, but we adopt a different tie-breaking strategy. First, for each uncolored part $P_i \in \mathcal{P}$, a candidate vertex is selected. The candidate is a vertex of minimum saturation degree in P_i, where the saturation degree corresponds to the number of distinct colors appearing in its neighborhood. Ties are broken analogously to the two-step heuristics, considering only uncolored neighbors. Finally, among all candidates, we select a vertex v of maximum saturation degree, breaking ties in favor of the candidate whose part $P_{i(v)}$ has minimum cardinality. If the color of v is not forced, we assign a color not used by its neighbors that invalidates the minimum number of vertices, avoiding colors that would make any part in $\mathcal{P}$ empty. If no such color exists, a new color is assigned to v, increasing the chromatic number.

6. A Branch-and-Price algorithm for solving DPCP

There are two predominant approaches in the literature for formulating graph coloring problems as integer linear programs (ILPs). The first consists of compact formulations that use decision variables to assign colors to vertices. The second relies on so-called *set covering formulations*, which involve an exponential number of binary decision variables, each corresponding to a stable set of the graph and representing a potential color class. The following formulation for the DPCP, proposed in Lucci et al. (2025), belongs to the latter class.

Given an instance $(G, \mathcal{P}, \mathcal{Q})$, let $\mathscr{S}(G)$ denote the collection of all stable sets of G. For each $S \in \mathscr{S}(G)$, we define a binary variable x_S that takes value 1 if and only if the stable set S is *active*, i.e., if it is selected as a color class in the solution. Then, the DPCP can be formulated as follows:

$$\text{(DPCP-SCF)} \quad \min \sum_{S \in \mathscr{S}(G)} x_S \tag{1}$$

$$s.t. \sum_{S \in \mathscr{S}(G): P \cap S \neq \emptyset} x_S \geq 1, \qquad P \in \mathcal{P}, \tag{2}$$

$$\sum_{S \in \mathscr{S}(G): Q \cap S \neq \emptyset} x_S \leq 1, \qquad Q \in \mathcal{Q}, \tag{3}$$

$$x \in \{0,1\}^{\mathscr{S}(G)}.$$

The objective function (1) minimizes the number of active stable sets (color classes) used in the solution. Constraints (2) ensure that, for each $P \in \mathcal{P}$, at least one vertex of P belongs to an active stable set and is therefore colored. Finally, constraints (3) ensure that each set $Q \in \mathcal{Q}$ intersects at most one active stable set, i.e., all colored vertices in Q receive the same color.

Observe that, for any singleton $\{v\} \in \mathcal{Q}$, the corresponding constraint in (3) can be removed from DPCP-SCF without affecting the optimal value. Indeed, removing such a constraint may yield a solution in which vertex v belongs to multiple color classes. In that case, removing v from all but one of these color classes produces a feasible solution to DPCP-SCF with the same objective value.

Hence, throughout the remainder of the paper we assume that constraints (3) are enforced only for sets $Q \in \mathcal{Q}$ with $|Q| \geq 2$, i.e., they are replaced by

$$\sum_{S \in \mathscr{S}(G): Q \cap S \neq \emptyset} x_S \leq 1, \quad Q \in \mathcal{Q},\ |Q| \geq 2. \tag{3'}$$

In the remainder of this section, we describe the main components of a B&P algorithm for solving DPCP-SCF. The pricing problem, the initialization procedure, and the branching scheme are presented in the following subsections. Throughout this section, we assume that an instance $(G, \mathcal{P}, \mathcal{Q})$ is given, with $G = (V, E)$, $\mathcal{P} = \{P_i : i \in [n]\}$, and $\mathcal{Q} = \{Q_j : j \in [m]\}$. We also assume that each set P_i induces a clique in G, since all missing edges are added during a preprocessing step.

For a comprehensive overview of the classical components of a B&P framework, we refer the reader to Desrosiers et al. (2026) and, in the context of graph coloring problems, to Mehrotra & Trick (1996); Furini et al. (2018).

6.1. Pricing problem

In the following, we assume without loss of generality that constraints (3) (or, equivalently, constraints (3')) in formulation DPCP-SCF have been multiplied by -1. Therefore, given a stable set $S \in \mathscr{S}(G)$, its associated column in DPCP-SCF is the 0/1 vector $x = (x^P, -x^Q) \in \{0,1\}^{n+m}$, where x^P and x^Q are the characteristic vectors of the sets $\mathcal{P}(S) = \{i(v) \in [n] : v \in S\}$ and $\mathcal{Q}(S) = \{j(v) \in [m] : v \in S\}$, respectively.

Observe that two distinct stable sets $S, S' \in \mathscr{S}(G)$ may correspond to the same column if $\mathcal{P}(S) = \mathcal{P}(S')$ and $\mathcal{Q}(S) = \mathcal{Q}(S')$. In our implementation, each column is linked to a single stable set, which allows colorings to be reconstructed from solutions of DPCP-SCF. Given a subset $\hat{\mathscr{S}} \subseteq \mathscr{S}(G)$, denote by DPCP-SCF($\hat{\mathscr{S}}$) the restriction of DPCP-SCF to the variables associated with stable sets in $\hat{\mathscr{S}}$. The pricing problem consists of determining, given an optimal solution x^* to the linear relaxation of DPCP-SCF($\hat{\mathscr{S}}$), whether there exists a column associated to a stable set in $\mathscr{S}(G) \setminus \hat{\mathscr{S}}$ with negative reduced cost.

Let $(\gamma^*, \mu^*) \in \mathbb{R}^{n+m}_+$ be an optimal solution to the dual of the linear relaxation of DPCP-SCF($\hat{\mathscr{S}}$). For any stable set $S \in \mathscr{S}(G)$, the reduced cost of the corresponding column is given by

$$\bar{c}_S = 1 - \left(\sum_{i \in \mathcal{P}(S)} \gamma_i^* - \sum_{j \in \mathcal{Q}(S)} \mu_j^* \right).$$

Hence, the pricing problem consists of finding a stable set $S \in \mathscr{S}(G) \setminus \hat{\mathscr{S}}$ such that $\bar{c}_S < 0$, or proving that no such set exists. Note that this condition is equivalent to

$$\sum_{i \in \mathcal{P}(S)} \gamma_i^* - \sum_{j \in \mathcal{Q}(S)} \mu_j^* > 1.$$

Proposition 6.1. *The pricing problem is NP-complete.*

Proof. It is immediate that the pricing problem belongs to NP. We prove NP-hardness by reduction from the decision version of the Weighted Stable Set Problem (WSSP).

An instance (G, w, k) of the WSSP consists of a graph $G = (V, E)$, a vector $w \in \mathbb{Z}^V_+$ of nonnegative integer vertex weights, and an integer $k \in \mathbb{Z}_+$. The problem asks whether G contains a stable set of weight at least k.

Consider the pricing problem defined on the same graph G, with partitions $\mathcal{P} = \{\{v\} : v \in V\}$ and $\mathcal{Q} = \{V\}$, and dual variables defined by $\gamma_v = w_v/(k-1)$ for all $v \in V$, and $\mu_V = 0$. Then, for any stable set $S \in \mathscr{S}(G)$, we have $\bar{c}_S = 1 - \frac{w(S)}{k-1}$. Since $w(S)$ and k are integers, there exists a stable set with $w(S) \geq k$ if and only if there exists a stable set with $\bar{c}_S < 0$. This completes the reduction. □

Using binary variables s_v, for $v \in V$, such that $s_v = 1$ if and only if $v \in S$, and binary variables w_j, for $j \in [m]$, such that $w_j = 1$ if and only if $Q_j \cap S \neq \emptyset$, the pricing problem can be formulated as the

following integer linear program:

$$\text{(DPCP-PRI)} \quad \max \sum_{i\in[n]} \gamma_i^* \sum_{v\in P_i} s_v - \sum_{j\in[m]} \mu_j^* w_j \tag{4}$$

$$\text{s.t.}$$

$$\sum_{v\in P_i} s_v \le 1, \qquad i \in [n], \tag{5}$$

$$s_v + \sum_{u\in N(v)\cap P_k} s_u \le 1, \qquad v \in V,\ k \in [n]\setminus\{i(v)\}, \tag{6}$$

$$s_v \le w_{j(v)}, \qquad v \in V, \tag{7}$$

$$s \in \{0,1\}^V,\ w \in \{0,1\}^m.$$

The objective function (4) ensures that an optimal solution S^* maximizes the dual contribution and, equivalently, minimizes the reduced cost $\bar{c}_{S^*} = 1 - (\sum_{i\in\mathcal{P}(S^*)} \gamma_i^* - \sum_{j\in\mathcal{Q}(S^*)} \mu_j^*)$.

Constraints (5) and (6) correspond to clique inequalities associated with the clique P_i, for $i \in [n]$, and the clique $\{v\} \cup (N(v) \cap P_k)$, for $v \in V$ and $k \in [n] \setminus \{i(v)\}$, respectively. Observe that each edge of G is covered by at least one of these cliques, which enforces that S^* is a stable set. Constraints (7) ensure that w_j is activated whenever S^* contains at least one vertex in Q_j.

6.1.1. Pricing heuristics

An instance of the pricing problem must be solved at every iteration of the column generation process, at each node of the branching tree. To reduce the associated computational effort, we employ three pricing algorithms. Only if none of them succeeds in finding a stable set with negative reduced cost, formulation DPCP-PRI is solved using a commercial ILP solver.

Heuristic pricing. We propose a greedy heuristic that constructs a stable set by repeatedly selecting a vertex of maximum weight. Unlike classical heuristics for the WSSP, where vertex weights are fixed, the weights in our pricing problem are adaptive, as they depend on previously selected vertices. Initially, each vertex v is assigned weight $\gamma^*_{i(v)} - \mu^*_{j(v)}$. Upon selecting a vertex v, all its neighbors are removed, and the weight of each remaining vertex $u \in Q_{j(v)}$ is updated to $\gamma^*_{i(u)}$.

This heuristic is further enhanced by a semi-greedy strategy based on a restricted candidate list (Feo & Resende, 1995), similar to the two-step heuristic previously proposed for the DPCP in Section 5.1. The resulting randomized variant is executed a fixed number of times, potentially generating multiple columns.

WSSP-based approach. By ignoring the adaptive behavior of the vertex weights and using fixed weights instead, the pricing problem can be approximated by the WSSP. After a stable set is obtained, its value is recomputed using the original pricing objective function.

Given a stable set $S \in \mathscr{S}(G)$, let $w(S)$ denote its weight under the objective function (4). Two fixed-weight variants are considered:

$$w_{\mathcal{PQ}}(S) = \sum_{v\in S} \left(\gamma^*_{i(v)} - \mu^*_{j(v)}\right) \quad \text{and} \quad w_{\mathcal{P}}(S) = \sum_{v\in S} \gamma^*_{i(v)}.$$

Observe that

$$w_{\mathcal{PQ}}(S) \le w(S) \le w_{\mathcal{P}}(S).$$

In the first approach, each vertex v is assigned weight $\gamma^*_{i(v)} - \mu^*_{j(v)}$, and vertices with negative weight are discarded. For any stable set S such that $w_{\mathcal{PQ}}(S) > 1$, it follows that $w(S) > 1$, and consequently a candidate column is generated. However, even when no such stable set exists, a stable set S^* maximizing $w_{\mathcal{PQ}}(S^*)$ may still be useful if it satisfies $w(S^*) > 1$.

Additionally, before evaluating the weight in the original pricing problem, the stable set can be greedily extended by adding vertices from any set $Q \in \mathcal{Q}$ that already contains selected vertices, since they have nonnegative contribution to the objective function.

In the second approach, each vertex v is assigned weight $\gamma^*_{i(v)}$, and a stable set S^* maximizing $w_{\mathcal{P}}(S^*)$ is computed. If $w_{\mathcal{P}}(S^*) \leq 1$, then $w(S) \leq 1$ for all stable sets $S \in \mathscr{S}(G)$, implying that no column with negative reduced cost exists and the column generation process can be safely terminated without solving the ILP formulation. Otherwise, the weight of S^* is evaluated in the original problem, and if $w(S^*) > 1$, a candidate column is obtained.

6.2. Initialization

To start the column generation process, an initial set of columns $\hat{\mathscr{S}} \subseteq \mathscr{S}(G)$ is required such that the linear relaxation of DPCP-SCF($\hat{\mathscr{S}}$) is feasible. For feasible instances of DPCP, the set of active columns in any feasible solution provides a valid choice for $\hat{\mathscr{S}}$. However, as discussed in Section 4, deciding feasibility of a general instance of DPCP is NP-complete. Consequently, initializing general instances of DPCP is more challenging than for instances arising from the VCP, PCP, or CFCP, for which feasible solutions can be easily constructed.

The initialization procedure consists of the following steps. First, we attempt to compute a heuristic solution using the approaches described in Section 5. If the heuristics fail, we resort to computing a basic feasible solution of LR, the linear relaxation of DPCP-SCF. A common approach for initializing linear programs consists of solving auxiliary problems with artificial variables. To this end, we solve the following parameterized linear program, which includes an additional variable t with objective coefficient given by a parameter $M \in \mathbb{Z}_+$:

$$
\begin{aligned}
(\mathrm{LR}^{\mathrm{M}}_{\mathrm{ext}}) \quad & \min \sum_{S \in \mathscr{S}(G)} x_S + Mt && && (8)\\
& \sum_{S \in \mathscr{S}(G): P \cap S \neq \emptyset} x_S + t \geq 1, && P \in \mathcal{P}, && (9)\\
& \sum_{S \in \mathscr{S}(G): Q \cap S \neq \emptyset} x_S \leq 1, && Q \in \mathcal{Q}, && (10)\\
& x \in [0,1]^{\mathscr{S}(G)}, \; t \in [0,1]. &&
\end{aligned}
$$

The following remark provides sufficient conditions to infer feasibility of LR:

Remark 6.1. *For any $M \in \mathbb{Z}_+$, let (x^*, t^*) be an optimal solution of LR^M_{ext} with optimal value z^*_{ext}.*

1. *If $t^* = 0$, then x^* is an optimal solution of LR.*
2. *Since $\min\{n, m\}$ is an upper bound on the optimal value of LR, it follows that if $t^* > 0$ and $z^*_{\mathrm{ext}} > \min\{n, m\}$, then LR is infeasible.*

The choice of parameter M plays an important role in practice. The arguments used in the proof of Lemma 4.1 in Lucci et al. (2024) can be adapted to show that there exists a value $M_0 \in \mathbb{R}$ such that,

for every $M \geq M_0$, an optimal solution of $\mathrm{LR}^{\mathrm{M}}_{\mathrm{ext}}$ satisfies one of the two conditions stated in Remark 6.1. Since excessively large values of M may lead to numerical instabilities, in our implementation we adopt the same value proposed in that work.

If neither condition of Remark 6.1 applies, that is, if $t^* > 0$ and $z^*_{\mathrm{ext}} \leq \min\{n, m\}$, we proceed with a standard two-phase initialization scheme. In the first phase, we solve the linear program $\mathrm{LR}_{\mathrm{ext}}$ obtained from $\mathrm{LR}^{\mathrm{M}}_{\mathrm{ext}}$ by replacing the objective function with the minimization of t. Observe that LR is feasible if and only if $\mathrm{LR}_{\mathrm{ext}}$ has optimal value zero. In that case, the optimal solution of $\mathrm{LR}_{\mathrm{ext}}$ is used to initialize LR, which is solved in the second phase.

6.3. Branching scheme

The branching scheme determines how the solution space is partitioned whenever a fractional solution is obtained at a node of the B&P tree. In the following, we extend to the DPCP a branching rule proposed by Furini et al. (2018) for the PCP.

First, observe that, if $|P| = 1$ holds for all $P \in \mathcal{P}$, then Proposition 4.1 implies that the DPCP instance can be transformed into a VCP instance. For PCP instances satisfying this condition, Furini et al. (2018) switch to the classical VCP branching scheme and continue the B&P procedure. Instead, we solve the current node using the B&P algorithm proposed in Held et al. (2012). Furthermore, we use the current upper bound (i.e., the value of the best solution found so far) to terminate this secondary B&P procedure as soon as its lower bound reaches or exceeds this value.

Our branching rule is applied only if there is at least one part $P \in \mathcal{P}$ with $|P| \geq 2$. It consists in selecting such a part P with $|P| \geq 2$ and containing the largest number of vertices that appear in the fractional solution x^*, that is, maximizing $\left|P \cap (\cup_{S \in \hat{\mathcal{S}}: x^*_S > 0} S)\right|$. Ties are broken by preferring smaller parts, and then by the index of the smallest part. After selecting a part P, we choose a vertex $v \in P$ maximizing its fractional assignment, i.e., $\sum_{S \in \hat{\mathcal{S}}: v \in S} x^*_S$. The solution space is then partitioned into two subspaces: those double-partition colorings in which v is colored and those in which v remains uncolored. Both cases lead to new DPCP instances, as described below.

1. If v is colored, then all vertices in $R = P_{i(v)} \setminus \{v\}$ need no longer be considered, since the requirement of coloring the part $P_{i(v)}$ is already satisfied. Therefore, they can be removed. This yields an instance $(G^1, \mathcal{P}^1, \mathcal{Q}^1)$ where $G^1 = G \setminus R$, $\mathcal{P}^1 = \{P^1_i : i \in [n]\}$ with $P^1_i = P_i$ for all $i \neq i(v)$ and $P^1_{i(v)} = \{v\}$, and $\mathcal{Q}^1 = \{Q_j \setminus R : j \in [m]\}$. Observe that $P^1_{i(v)}$ is now a singleton, and Lemma 4.1 can be applied.

2. If v is uncolored, then it can be removed, and another vertex in $P_{i(v)}$ must be colored instead. This leads to an instance $(G^2, \mathcal{P}^2, \mathcal{Q}^2)$ where $G^2 = G \setminus \{v\}$, $\mathcal{P}^2 = \{P^2_i : i \in [n]\}$ with $P^2_i = P_i$ for all $i \neq i(v)$ and $P^2_{i(v)} = P_{i(v)} \setminus \{v\}$, and $\mathcal{Q}^2 = \{Q^2_j : j \in [m]\}$ with $Q^2_j = Q_j$ for all $j \neq j(v)$ and $Q^2_{j(v)} = Q_{j(v)} \setminus \{v\}$.

Since the cardinality of the selected part P decreases in both child nodes, after applying the branching rule a finite number of times, the case $|P| = 1$, for all $P \in \mathcal{P}$ is eventually reached at the leaves of the enumeration tree.

For DPCP instances arising from PCP and CFCP, each child node again corresponds to an instance of the PCP or the CFCP, respectively, which are known to be always feasible. In contrast, this branching scheme may generate infeasible child nodes for general DPCP instances, even when the original instance is feasible, as illustrated in the following example.

Example 6.1. *Let $G = P_3$ be the path graph with vertices v_1, v_2, v_3, and let $\mathcal{P} = \{\{v_1\}, \{v_2, v_3\}\}$ and $\mathcal{Q} = \{\{v_1, v_2\}, \{v_3\}\}$. Clearly, this instance is feasible, since v_1 and v_3 can be assigned different colors. If we branch on v_2, then the resulting instance $(G^1, \mathcal{P}^1, \mathcal{Q}^1)$ is infeasible. Similarly, branching on v_3 leads to an infeasible instance $(G^2, \mathcal{P}^2, \mathcal{Q}^2)$. In both cases, vertices v_1 and v_2 are forced to receive the same color despite being adjacent, leading to a contradiction.*

7. Experimental setup and Branch-and-Price configuration

In this section, we describe the experimental setup and report preliminary experiments conducted to configure the B&P algorithm. The computational experiments were carried out on a computer equipped with Ubuntu 22.04.5 LTS, using a single thread of an Intel Core i5-3230M CPU at 2.60 GHz and 6 GB of RAM. The repository, including the source code, instances, results, and analysis, is available at `https://github.com/maurolucci/dpcp`.

7.1. Instances

We generated random DPCP instances based on the number of vertices $|V|$, the edge probability p, and the cardinalities $n = |\mathcal{P}|$ and $m = |\mathcal{Q}|$. First, an Erdős–Rényi graph with parameters $|V|$ and p is generated. Then, each vertex is assigned uniformly at random to a part of $\mathcal{P}$ and of $\mathcal{Q}$, while preventing empty parts in the partitions.

For CFCP, three datasets exhibiting different structural properties are considered. First, we generate hypergraphs arising from frequency assignment problems with circular coverage areas. The generator takes as input the number of antennas (c) and their coverage radius (r). The antennas are placed uniformly at random within a 100×100 square region. The resulting hypergraph has one vertex for each antenna, while each hyperedge represents the set of antennas covering a connected region of the coverage area (see Figure 1). To identify these regions, we discretize the plane using a regular grid with a spacing of 0.1 between adjacent points, and compute which antennas are reachable from each point of the grid. The other two sets of CFCP instances correspond to hypergraphs induced by the open and closed neighborhoods of the vertices of some DIMACS instances (`https://mat.tepper.cmu.edu/COLOR/instances.html`).

As shown in Appendix A.1, for instances arising from the PCP and the LCP, formulation DPCP-SCF reduces to the set covering formulations proposed in Furini et al. (2018); Lucci et al. (2024). Consequently, under the same algorithmic choices and computational environment, experiments on PCP and LCP instances would provide limited additional insight.

In summary, the performance of the algorithms will be computationally evaluated on the following sets of instances:

DPCP-ER-1 contains three random DPCP instances for each combination of the following parameter values: $|V| = 150$, $p \in \{0.25, 0.5, 0.75\}$, $n \in \{0.2|V|, 0.3|V|\}$, and $m \in \{0.2|V|, 0.3|V|\}$. This results in a total of 36 instances.

DPCP-ER-2 is analogous to DPCP-ER-1, but with $|V| = 110$.

CFCP-FA contains five random CFCP instances of frequency assignment for each combination of the following parameter values: $(c, r) \in \{(9, 50), (12, 35), (18, 20)\}$. This results in a total of 15 instances.

heuristic	#solved	value	time (ms)	≥best (%)	best (%)
greedy 1-step	16	22.8	3.51	33	11
greedy 2-step (DEG)	33	10.5	1.04	36	36
greedy 2-step (QDEG)	31	12.0	1.29	39	8
greedy 2-step (SQDEG)	32	10.8	1.57	67	36

Table 1: Average results of the greedy heuristics for the DPCP-ER-1 dataset.

CFCP-ON contains the CFCP instances associated with the open neighborhoods of the DIMACS instances. In this dataset, isolated vertices are first removed from the DIMACS instances to guarantee feasibility. Only instances whose conflict graphs have at most 1000 vertices are considered, resulting in a total of 9 instances.

CFCP-CN is analogous to CFCP-ON, but with closed neighborhoods.

The next two sections present preliminary experiments on the DPCP-ER-1 dataset aimed at fine-tuning the B&P algorithm.

7.2. Evaluation and selection of DPCP heuristics

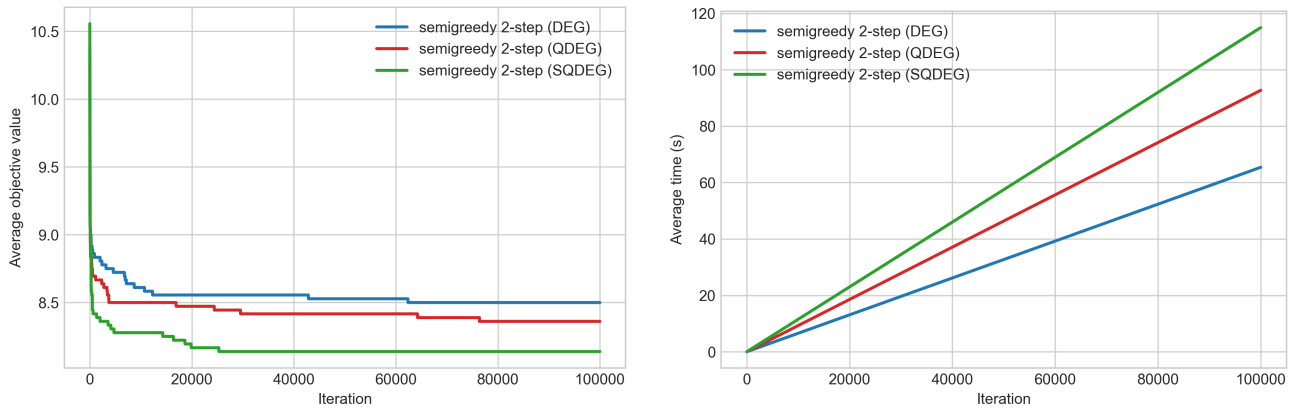
We begin by evaluating the performance of the DPCP heuristics presented in Section 5: the one-step greedy heuristic, and the two-step greedy heuristic with the three vertex-selection criteria DEG, QDEG, and SQDEG.

Results are reported in Table 1, where each row summarizes the performance of a heuristic over all instances in the dataset, including the number of instances for which a solution was found, the average objective value, and the average execution time. Column "≥best" reports the percentage of instances in which the heuristic finds the best solution among all heuristics. Since several heuristics may find the same best solution, column "best" breaks ties using the execution time, so exactly one heuristic is counted for each instance. When a heuristic fails to find a solution, we assign as objective value the minimum cardinality of the two partitions, providing a valid upper bound for feasible instances and ensuring a fair comparison in terms of the average objective value.

The two-step greedy heuristics succeed in finding solutions for nearly all instances. In contrast, the one-step greedy heuristic fails to find solutions for roughly half of the instances and takes more than three times as long. Among the vertex-selection criteria, SQDEG achieves the best objective value in 67% of the instances, nearly twice as often as any other criterion. However, SQDEG is 34% slower than DEG and 18% slower than QDEG. When execution time is used to break ties, DEG emerges as a competitive alternative, achieving the best performance in 36% of the instances, tied with SQDEG. Overall, these results demonstrate that the two-step approach consistently outperforms the one-step approach in terms of both solution quality and computational efficiency.

These results motivated the evaluation of semigreedy variants of the two-step heuristics, using $\alpha = 0.15$ to define the restricted candidate list. We analyze the average best objective value and execution time as a function of the number of iterations, as shown in Figures 6a and 6b.

Introducing randomness into the two-step heuristics quickly yields improvements in solution quality of more than 20% compared with their greedy counterparts. Among the three criteria, SQDEG consistently



(a) Average best objective value

(b) Average cumulative execution time

Figure 6: Results of the semigreedy heuristics for the DPCP-ER-1 dataset.

provides the best solutions. The improvement in solution quality becomes progressively smaller as the number of iterations increases, whereas the running time grows almost linearly with the number of iterations. For SQDeg, all the improvement is obtained within the first 30,000 iterations. All heuristics find solutions for the 34 feasible instances of the dataset within the first 300 iterations, while the remaining two instances are correctly identified as infeasible during preprocessing.

Based on these results, we adopt the two-step approach with SQDeg. The greedy version of the heuristic is used to quickly initialize the linear relaxation at every node of the B&P tree, whereas the semi-greedy version is executed only once at the root node to obtain a high-quality initial upper bound that promotes earlier pruning of the search tree. The latter runs for 40,000 iterations or 1 minute, whichever comes first.

7.3. Evaluation and selection of pricing algorithms

As a second experiment, we evaluate the pricing algorithms presented in Section 6.1. To this end, we solve the linear relaxation at the root node of each instance, with a time limit of 15 minutes. We generate the initial columns as described in Section 6.2. More specifically, we use a greedy solution obtained by the two-step greedy heuristic using the SQDeg criterion. If the heuristic fails to produce a feasible solution, a dummy variable is used instead, with objective coefficient $M = 1000$, as suggested in Lucci et al. (2024).

Our baseline pricing algorithm consists of solving DPCP-PRI directly with an ILP solver. We incorporate an early-stopping mechanism that terminates the search as soon as a stable set of weight at least 1.1 is found. This avoids the additional effort of proving optimality once a sufficiently attractive column has been identified. Starting from this baseline, the remaining pricing algorithms are incorporated incrementally to evaluate the contribution of each enhancement.

First, we enable heuristic pricing by executing the randomized heuristic until either 20 candidate stable sets are found or 100 iterations are reached, with $\alpha = 0.2$ for the restricted candidate list. If no columns are found, we resort to exact pricing.

Subsequently, the two WSSP-based pricing algorithms are activated, separately and jointly. These algorithms are applied after heuristic pricing fails and before exact pricing is invoked. Both rely on the exact algorithm of Held et al. (2012) for WSSP (available at `https://github.com/heldstephan/`

config.	#solved	pricing time (s)					#added cols				
		total	heur	WSSP$_{\mathcal{P}}$	WSSP$_{\mathcal{PQ}}$	exact	total	heur	WSSP$_{\mathcal{P}}$	WSSP$_{\mathcal{PQ}}$	exact
i	22	603.1	–	–	–	603.1	315	–	–	–	315
ii	28	449.2	1.2	–	–	448.0	551	512	–	–	38
iii	32	279.3	1.3	1.7	–	276.3	570	500	47	–	21
iv	29	418.4	1.2	–	2.7	414.5	554	512	–	6.6	35
v	32	259.3	1.3	1.6	1.3	255.1	571	500	48	1.5	20

Table 2: Average results of the pricing algorithms on the DPCP-ER-1 linear relaxations.

`exactcolors`), which combines a greedy heuristic with combinatorial enumeration. We refer to these pricing algorithms as WSSP$_{\mathcal{P}}$ and WSSP$_{\mathcal{PQ}}$, respectively, following the notation of their corresponding weight functions.

In Table 2, each row summarizes the average performance of a pricing configuration over all instances in the dataset. The first column indicates the configuration, while the second reports the number of linear relaxations solved within the time limit. The remaining columns report the average total pricing time and the average total number of generated columns, both broken down by pricing algorithm. The enabled pricing algorithms for each configuration (i)–(v) can be identified directly from the table, where entries marked "–" indicate disabled algorithms.

All pricing algorithms improve the baseline, with heuristic pricing generating most of the added columns and reducing those generated by exact pricing from 315 to fewer than 40. Nevertheless, most of the pricing time is still spent in exact pricing, suggesting that further improvements should focus on this component. Overall, combining heuristic pricing with both WSSP-based pricing algorithms yields the best performance, reducing the average pricing time by 57% (from 603.1 s to 259.3 s) while increasing the number of solved instances from 22 to 32. Thus, configuration (v) is adopted throughout the B&P algorithm.

Preliminary tests suggested the convenience of keeping a private column pool at each node of the B&P tree, which is scanned before any pricing algorithm is invoked. At the root node, the pool is initialized with the stable sets generated during the execution of the semi-greedy two-step heuristic. At each non-root node, the set of columns generated during initialization is augmented with the valid columns having positive value in the optimal solution of the parent node. The remaining inherited columns from the parent node are stored in the node's private pool. To limit memory consumption, each pool stores at most 1000 columns.

In what follows, the B&P tree is explored using a best-bound strategy, selecting for branching the node with the lowest linear relaxation value. Each node of the tree is preprocessed by repeatedly applying Lemmas 4.1 and 4.3 until no further reduction is possible. This reduces the instance size and enables early infeasibility detection.

8. Computational evaluation of DPCP solvers

In this section, we evaluate two approaches for solving the DPCP: the B&P algorithm, using the settings established in the previous section, and the direct solution of the compact ILP formulation proposed in Lucci et al. (2025). For the latter, we use Gurobi v12.0.3 with its default parameters. The

p	solver	#status			time	abs.
		opt	infeas	tilim	(s)	gap
0.25	b&p	7	1	4	1281	0.4
	gurobi	11	1	0	8	0.0
0.50	b&p	8	2	2	1275	0.2
	gurobi	5	2	5	1580	0.6
0.75	b&p	8	2	2	763	0.2
	gurobi	1	2	9	2867	2.7

Table 3: Average results by edge probability on the DPCP-ER-2 dataset.

evaluation is conducted on DPCP-ER-2 dataset and the three datasets containing CFCP instances, with a time limit of one hour per instance. To ensure a fair comparison, both approaches start from the same preprocessed instance and are initialized with the same solution generated by the two-step semi-greedy heuristic, which serves as an initial incumbent for the B&P algorithm and as a warm start for the compact formulation.

8.1. Performance on random DPCP instances

Table 3 presents the results for the DPCP-ER-2 dataset, where each row reports the average performance over instances with the same edge probability p. The columns report the solver (B&P or Gurobi), the total number of instances solved to optimality ("opt"), proven infeasible ("infeas"), or terminated due to the time limit ("tilim"), the average execution time (excluding the time spent by the initial execution of the DPCP heuristic), and the absolute optimality gap. The latter is averaged over all feasible instances, assigning a gap of 0 to instances solved to optimality.

As shown in the table, the performance is strongly affected by the edge probability p. While B&P improves as p increases, the compact formulation exhibits the opposite behavior. In particular, B&P clearly outperforms the compact formulation on high-density instances, proving optimality for 8 instances compared with only 1 when $p = 0.75$. Conversely, the compact formulation performs better on low-density instances, solving 11 instances to optimality compared with 7 for B&P when $p = 0.25$, while requiring only 8 seconds on average versus 1281 seconds for B&P. For medium-density instances ($p = 0.5$), B&P has a slight advantage over the compact formulation, proving optimality for 8 instances against 5.

Additionally, although not shown in the table, all five infeasible instances in this dataset have $m = 0.2|V|$, the smallest value considered, suggesting a higher likelihood of generating infeasible instances for smaller values of m. In all cases, infeasibility was detected in the preprocessing phase. However, as mentioned in Example 6.1, infeasible nodes may still arise during the B&P search on feasible instances. In practice, this occurs very rarely: across all instances, only 334 of the 8,888 generated nodes (3.76%) required dummy initialization, and only 3 were proven to have an infeasible relaxation (0.03% of all nodes). Therefore, additional infeasibility detection procedures are not a priority for this dataset.

To further analyze performance, Table 4 presents the instance-by-instance results, excluding infeasible instances. The first column contains the instance name, which encodes the number of vertices $|V|$, the edge probability p, and the cardinalities n and m of $\mathcal{P}$ and $\mathcal{Q}$, respectively, and the instance seed (e.g., `r_V110_p0.25_n22_m22_i0`). The column "hval" reports the value of the initial solution obtained by the DPCP heuristic. For each solver, we report the execution time, the number of generated nodes, and the

final lower and upper bounds. For B&P, we additionally report the value of the root linear relaxation ("rval"), as well as the pricing time and the number of generated columns, including columns retrieved from the pool. These pricing statistics are reported separately for the root node and averaged over the remaining nodes. We use boldface to indicate the best result between the two algorithms, with execution time as the primary criterion. In case of a tie at the time limit, the upper bound is used as the secondary criterion, followed by the lower bound.

The individual results confirm the trends previously observed for low- and high-density graphs, and provide further insight into medium-density graphs, where the relative performance of the two approaches depends on n. When $\mathcal{P}$ has fewer parts, the compact formulation is consistently faster than B&P, whereas when $\mathcal{P}$ has more parts, B&P solves three instances to optimality compared with none for the compact formulation.

Moreover, these results further support the effectiveness of the DPCP heuristic on random instances and highlight the benefits of the proposed column management strategy over column pooling and inheritance. Specifically, the initial execution of the DPCP heuristic finds solutions within one unit of the optimum for 30 of the 31 instances, and finds optimal solutions for 16 of them. Furthermore, on average, pricing at non-root nodes requires 68.6% less time and generates 81.5% fewer columns than pricing at the root node.

8.2. Performance on CFCP instances

In this section, we evaluate the algorithms on CFCP instances with different structural characteristics, using the CFCP-FA, CFCP-ON, and CFCP-CN datasets, to determine whether additional structural factors, beyond those identified on random DPCP instances, influence algorithmic performance. Since these instances involve substantially larger conflict graphs, we reduce the limit on successive pricing heuristic calls from 100 to 10. In addition, the time limit is set to 15 minutes for CFCP-FA instances to avoid memory issues, while remaining at 1 hour for the other datasets.

Table 5 reports the results for each frequency assignment instance in the CFCP-FA dataset. The first column lists the instance name, which encodes the number of antennas c, the radius r, and the instance seed (e.g., `freq_c18_r20_i0`). The following columns describe the associated conflict graph by reporting its number of vertices $|V|$, density p, the cardinalities n and m of $\mathcal{P}$ and $\mathcal{Q}$, and the number of colors in the initial heuristic solution ("hval"). The remaining columns report, for each solver, the execution time (excluding the time spent by the initial execution of the DPCP heuristic), the number of explored nodes, and the final lower and upper bounds.

Overall, the results are consistent with the trends observed for the random DPCP instances. The compact formulation remains the preferred approach for low-density graphs, whereas B&P performs better on medium- and high-density graphs. A notable difference is observed for the compact formulation at the largest coverage radius, which generates dense conflict graphs: unlike in the previous experiment, it solves all instances to optimality. Nevertheless, for these instances, B&P still achieves substantially shorter running times in all but one instance.

An interesting characteristic of these instances is their small chromatic number: all but one require only three colors. The DPCP heuristic finds an optimal solution in 5 out of the 15 instances, a lower proportion than for the random DPCP instances (16 out of 31, excluding infeasible ones). This suggests that the heuristic could be further improved for instances with this specific structure.

instance	hval	b&p									gurobi			
		time (s)	nodes	rval	lb	ub	pricing time (s)		#added cols		time (s)	nodes	lb	ub
							root	non-root	root	non-root				
r_V110_p0.25_n22_m22_i0	2	6.6	1	1.6	2.0	2	6.6	–	420	–	**<0.1**	0	2.0	2
r_V110_p0.25_n22_m22_i1	2	34.9	1	1.5	2.0	2	34.9	–	474	–	**0.2**	1	2.0	2
r_V110_p0.25_n22_m22_i2	3	833.1	193	1.6	2.0	2	12.9	4.3	505	100	**0.6**	1	2.0	2
r_V110_p0.25_n22_m33_i0	2	9.2	1	1.6	2.0	2	9.1	–	460	–	**<0.1**	0	2.0	2
r_V110_p0.25_n22_m33_i1	2	2.0	1	1.6	2.0	2	1.9	–	475	–	**<0.1**	0	2.0	2
r_V110_p0.25_n22_m33_i2	2	9.5	1	1.6	2.0	2	9.5	–	455	–	**0.1**	1	2.0	2
r_V110_p0.25_n33_m22_i1	4	*tilim*	234	2.6	3.0	4	39.3	15.2	802	166	**10.4**	667	3.0	3
r_V110_p0.25_n33_m22_i2	4	*tilim*	119	2.6	3.0	4	78.9	29.7	844	168	**75.0**	5804	4.0	4
r_V110_p0.25_n33_m33_i0	4	*tilim*	363	2.6	3.0	4	31.1	9.8	733	148	**2.8**	569	3.0	3
r_V110_p0.25_n33_m33_i1	3	77.8	1	2.4	3.0	3	77.7	–	776	–	**1.7**	4	3.0	3
r_V110_p0.25_n33_m33_i2	4	*tilim*	304	2.5	3.0	4	40.4	11.7	686	144	**9.9**	652	3.0	3
r_V110_p0.5_n22_m22_i1	4	765.6	97	2.8	4.0	4	21.5	7.7	384	78	**522.9**	39595	4.0	4
r_V110_p0.5_n22_m22_i2	4	503.8	41	2.9	4.0	4	25.6	11.9	373	77	**145.0**	4964	4.0	4
r_V110_p0.5_n22_m33_i0	4	91.5	35	2.9	4.0	4	9.5	2.4	346	62	**21.3**	3899	4.0	4
r_V110_p0.5_n22_m33_i1	4	899.9	359	2.8	4.0	4	7.7	2.5	389	63	**245.8**	18697	4.0	4
r_V110_p0.5_n22_m33_i2	4	1216.1	419	2.7	3.0	3	6.3	2.9	364	64	**35.9**	1280	3.0	3
r_V110_p0.5_n33_m22_i1	7	**2114.4**	349	5.2	6.0	6	27.9	6.0	468	98	*tilim*	38099	5.0	6
r_V110_p0.5_n33_m22_i2	7	*tilim*	259	5.2	6.0	7	61.6	13.6	491	97	*tilim*	73635	4.0	**6**
r_V110_p0.5_n33_m33_i0	6	**1326.7**	243	4.7	6.0	6	12.1	5.4	497	88	*tilim*	108460	5.0	6
r_V110_p0.5_n33_m33_i1	6	*tilim*	466	4.6	5.0	6	33.1	7.6	460	84	*tilim*	100742	5.0	6
r_V110_p0.5_n33_m33_i2	6	**1190.7**	217	4.7	6.0	6	10.1	5.5	510	92	*tilim*	112387	5.0	6
r_V110_p0.75_n22_m22_i1	6	**155.9**	47	4.8	6.0	6	6.8	3.2	235	46	*tilim*	120435	5.0	6
r_V110_p0.75_n22_m22_i2	6	**288.2**	57	4.8	6.0	6	16.6	4.8	287	49	*tilim*	103374	4.0	6
r_V110_p0.75_n22_m33_i0	6	**236.6**	145	4.7	6.0	6	6.9	1.6	259	33	*tilim*	258464	5.0	6
r_V110_p0.75_n22_m33_i1	6	**145.1**	69	4.6	5.0	5	4.4	2.1	264	38	2005.0	26608	5.0	5
r_V110_p0.75_n22_m33_i2	6	**219.5**	115	4.6	5.0	5	5.7	1.9	281	36	*tilim*	116335	4.0	5
r_V110_p0.75_n33_m22_i1	–	**18.3**	15	10.8	12.0	12	7.9	0.7	322	75	*tilim*	1855	8.0	12
r_V110_p0.75_n33_m22_i2	12	**346.4**	71	9.7	11.0	11	17.5	4.7	421	66	*tilim*	10884	6.0	12
r_V110_p0.75_n33_m33_i0	10	*tilim*	2465	8.1	**9.0**	10	3.2	1.4	311	46	*tilim*	18631	6.0	10
r_V110_p0.75_n33_m33_i1	10	*tilim*	1918	8.0	**9.0**	10	13.2	1.8	375	50	*tilim*	7451	6.0	10
r_V110_p0.75_n33_m33_i2	10	**546.1**	275	8.2	9.0	9	10.7	1.9	345	57	*tilim*	7406	6.0	10

Table 4: Per-instance results on the DPCP-ER-2 dataset.

instance	$\|V\|$	p	n	m	hval	time (s)		nodes		lb		ub	
						b&p	gurobi	b&p	gurobi	b&p	gurobi	b&p	gurobi
`freq_c18_r20_i0`	257	0.22	94	18	4	*tilim*	**324.2**	22799	2553	3.0	4.0	4	4
`freq_c18_r20_i1`	212	0.25	86	18	4	**24.9**	86.4	503	3	3.0	3.0	3	3
`freq_c18_r20_i2`	288	0.47	96	18	4	*tilim*	**214.1**	9166	585	3.0	3.0	4	3
`freq_c18_r20_i3`	231	0.27	87	18	4	*tilim*	**60.6**	15707	1	3.0	3.0	4	3
`freq_c18_r20_i4`	265	0.33	91	18	4	*tilim*	**51.9**	15044	1	3.0	3.0	4	3
`freq_c12_r35_i0`	570	0.65	111	12	5	**231.3**	*tilim*	187	1	3.0	2.0	3	4
`freq_c12_r35_i1`	229	0.45	68	12	4	**22.2**	110.2	261	253	3.0	3.0	3	3
`freq_c12_r35_i2`	337	0.55	85	12	4	**27.9**	656.3	117	1	3.0	3.0	3	3
`freq_c12_r35_i3`	449	0.58	104	12	4	*tilim*	*tilim*	2076	1	3.0	2.0	4	**3**
`freq_c12_r35_i4`	269	0.46	75	12	4	**24.6**	330.3	183	904	3.0	3.0	3	3
`freq_c9_r50_i0`	297	0.72	62	9	3	**0.7**	707.3	3	1	3.0	3.0	3	3
`freq_c9_r50_i1`	244	0.68	56	9	3	185.7	**168.6**	1265	124	3.0	3.0	3	3
`freq_c9_r50_i2`	205	0.69	49	9	4	**6.7**	223.5	103	46	3.0	3.0	3	3
`freq_c9_r50_i3`	276	0.70	62	9	3	**0.6**	429.0	3	36	3.0	3.0	3	3
`freq_c9_r50_i4`	258	0.71	56	9	3	**2.7**	194.9	21	522	3.0	3.0	3	3

Table 5: Per-instance results on the CFCP-FA dataset.

Tables 6a and 6b report the results for the CFCP instances associated with open and closed neighborhoods, respectively. They follow the same format as the previous table. These results also support the observed relationship between graph density and algorithm performance. The conflict graphs in these instances have densities ranging from 0.38 to 0.75, corresponding to low-medium to high-medium density graphs. In this range, B&P achieves the best running times in most cases, with the only exceptions being the open-neighborhood instance `jean` and `queen6_6`. Both algorithms reach the time limit or memory limit on `queen7_7`, although B&P obtains a stronger lower bound for the open-neighborhood instance. The DPCP heuristic finds an optimal solution for 4 of the 9 open-neighborhood instances and 3 of the 9 closed-neighborhood instances. Gurobi explores only a single node in all but one of the instances it solves, highlighting its strong heuristic capabilities, but this is not enough to outperform B&P in running time.

9. Conclusions and future work

The Double-Partition Coloring Problem (DPCP) was introduced in Lucci et al. (2025) as a generalization of several coloring problems on graphs and hypergraphs. In this work, we complete the proofs establishing that the List Coloring Problem (LCP) and the Conflict-Free Coloring Problem (CFCP) reduce to the DPCP. In addition, we develop the details of a branch-and-price (B&P) algorithm for solving a set covering formulation of the DPCP. Together with the compact formulation presented in our previously mentioned work, which is solved by a state-of-the-art IP solver in its standard configuration, these exact approaches provide an effective framework for addressing a wide range of coloring problems. To the best of our knowledge, they are the first exact methods to be proposed and computationally tested on CFCP instances.

The development of a B&P scheme revealed several structural properties of the DPCP, two of which are worth highlighting here. First, the feasibility of a DPCP instance is characterized in terms of the stability number of a polynomial-sized auxiliary graph. Second, general DPCP instances cannot be solved independently on each connected component of the underlying graph. We characterize the instances for

instance	$\|V\|$	p	n	m	hval	time (s)		nodes		lb		ub	
						b&p	gurobi	b&p	gurobi	b&p	gurobi	b&p	gurobi
david	803	0.61	78	87	3	**12.5**	37.2	3	1	2.0	2.0	2	2
huck	599	0.46	72	74	6	**16.7**	622.1	11	1	2.0	2.0	2	2
jean	495	0.38	66	80	4	1738.7	**306.6**	1365	645	3.0	3.0	3	3
myciel3	40	0.39	11	11	2	<0.1	<0.1	1	1	2.0	2.0	2	2
myciel4	142	0.40	23	23	2	**0.1**	1.2	1	1	2.0	2.0	2	2
myciel5	472	0.39	47	47	2	**3.1**	61.0	1	1	2.0	2.0	2	2
queen5_5	320	0.71	25	25	2	**0.5**	53.2	1	1	2.0	2.0	2	2
queen6_6	580	0.66	36	36	3	2669.1	**1161.5**	41	1	2.0	2.0	2	2
queen7_7	952	0.61	49	49	4	*tilim*	*memlim*	11	1	**2.0**	1.0	4	4

(a) Open neighborhoods

instance	$\|V\|$	p	n	m	hval	time (s)		nodes		lb		ub	
						b&p	gurobi	b&p	gurobi	b&p	gurobi	b&p	gurobi
david	791	0.63	74	87	3	**155.5**	330.4	5	1	2.0	2.0	2	2
huck	455	0.52	49	74	4	**8.2**	123.4	7	1	2.0	2.0	2	2
jean	477	0.40	66	80	4	**93.5**	164.2	55	1	2.0	2.0	2	2
myciel3	51	0.56	11	11	2	**<0.1**	0.1	1	1	2.0	2.0	2	2
myciel4	165	0.50	23	23	3	**3.5**	7.2	19	1	2.0	2.0	2	2
myciel5	519	0.45	47	47	3	**202.7**	239.1	49	1	2.0	2.0	2	2
queen5_5	345	0.75	25	25	2	**0.6**	66.3	1	1	2.0	2.0	2	2
queen6_6	616	0.69	36	36	2	**6.1**	432.1	1	1	2.0	2.0	2	2
queen7_7	1001	0.63	49	49	4	*tilim*	*memlim*	1	1	–	**1.0**	4	4

(b) Closed neighborhoods

Table 6: Per-instance results on the CFCP-ON and CFCP-CN datasets

which such a decomposition is possible.

Greedy and semi-greedy heuristics for the DPCP were proposed following one-step and two-step schemes. In our experimental setting, the two-step heuristics outperformed the one-step heuristic, finding good solutions for nearly all feasible instances with average running times below 2 ms. Introducing randomness improved the solution values by more than 20% within a few seconds. When used to initialize B&P, one semi-greedy two-step heuristic found solutions within one unit of the optimum for 30 of 31 feasible instances and optimal solutions for half of these instances. These results highlight the effectiveness of the proposed heuristics both as stand-alone methods and as initialization procedures for B&P.

After showing that the pricing problem is NP-hard, we propose an integer programming formulation and three heuristic solution strategies. In the computational experiments, the best overall performance was obtained when all three pricing heuristics were activated.

The performance of the B&P algorithm was compared with the direct solution of the compact formulation (using a state-of-the-art solver) on one set of randomly generated DPCP instances and on three sets of CFCP instances (frequency assignment, open neighborhoods and closed neighborhoods). The results show that both approaches complement each other on random DPCP instances and frequency assignment instances: the B&P tends to perform better when the edge density of the graph is high, while the compact formulation performs better on low-density graphs. For CFCP instances arising from open and closed neighborhoods in graphs, the B&P algorithm outperforms the compact formulation.

These approaches could be extended to effectively solve a broader class of DPCP instances through further work on specific aspects of the B&P algorithm.

Improving the branching scheme could have a significant impact on the performance of the algorithm. One possible research direction is to introduce an alternative branching rule, as proposed by Furini et al. (2018) for the PCP, which covers the cases when all parts in the first partition of the instance have cardinality equal to one, where our current approach resorts to solving a VCP instance with a separate B&P procedure. Moreover, a non-dichotomous rule combining our current rule with the classical VCP branching rule could be explored as an alternative.

A further research direction consists of improving the pricing procedure. This could be achieved, for instance, by strengthening the pricing ILP formulation or developing a combinatorial enumeration algorithm, as proposed by Held et al. (2012) for the VCP.

Appendix A. DPCP-SCF vs. extended formulations for PCP and LCP

In this appendix, we characterize the relationship between DPCP-SCF and previously proposed set covering formulations for the PCP and the LCP.

Proposition Appendix A.1. *DPCP-SCF is equivalent to the formulation* ILP^{E} *proposed in Furini et al. (2018) for the PCP, and to the formulation WLCP-SC proposed in Lucci et al. (2024) for the LCP, in the following sense:*

(a) Let $I = (G, \mathcal{P})$ *be an instance of the PCP, with* $G = (V, E)$*, and let* $I' = (G, \mathcal{P}, \mathcal{Q})$ *be the corresponding DPCP instance with* $\mathcal{Q} = \{\{v\} : v \in V\}$*. Then, formulation* ILP^{E} *for* I *coincides with formulation DPCP-SCF for* I'*.*

(b) Let $I = (G, C, \{L_v : v \in V\})$ *be an instance of the LCP, with* $G = (V, E)$*, and assume that* I *contains no indistinguishable colors, i.e., that the sets* $V^k = \{v \in V : k \in L_v\}$ *are distinct for all* $k \in C$*. Let* I' *be the corresponding DPCP instance constructed as described in Section 3. Then, formulation WLCP-SC for* I *coincides with formulation DPCP-SCF for* I'*, provided that all color weights are equal to one.*

Proof. For part (a), observe that $|Q| = 1$ holds for every $Q \in \mathcal{Q}$ in instance I'. Therefore, the family of constraints (3') is empty, and DPCP-SCF coincides with the formulation ILP^{E} proposed in Furini et al. (2018).

For part (b), following the construction introduced in Section 3, let I' denote the DPCP instance $(\tilde{G}, \mathcal{P}, \mathcal{Q})$, where $\tilde{G} = (\tilde{V}, \tilde{E})$ is the graph with vertex set $\tilde{V} = \{(v, k) : v \in V,\ k \in L_v\}$, and edge set $\tilde{E}$ such that $(v_1, k_1)(v_2, k_2) \in \tilde{E}$ if and only if either $v_1 v_2 \in E$ or $k_1 \neq k_2$. Moreover, $\mathcal{P} = \{P_v : v \in V\}$ and $\mathcal{Q} = \{Q_k : k \in C\}$, where $P_v = \{(v', k) \in \tilde{V} : v' = v\}$ and $Q_k = \{(v, k') \in \tilde{V} : k' = k\}$.

For each $k \in C$, let $G_k = G[V^k]$ be the subgraph of G induced by V^k. Since I has no indistinguishable colors, the set K of equivalence classes introduced in Lucci et al. (2024) coincides with C, and $m(k) = 1$ for every $k \in C$. Consequently, the decision variables of WLCP-SC are indexed by $X = \{(S, k) : k \in C,\ S \in \mathscr{S}(G_k)\}$. We first establish a bijection between X and $\mathscr{S}(\tilde{G})$.

Let $\tilde{S} \in \mathscr{S}(\tilde{G})$ with $\tilde{S} \neq \emptyset$. Then there exists a unique $k \in C$ such that $\tilde{S} \subseteq Q_k$. Indeed, if $(v_1, k_1), (v_2, k_2) \in \tilde{S}$ with $k_1 \neq k_2$, then (v_1, k_1) and (v_2, k_2) are adjacent in $\tilde{G}$, contradicting the stability of $\tilde{S}$. Define $S = \{v \in V : (v, k) \in \tilde{S}\}$. Then $S \subseteq V^k$ and $S \in \mathscr{S}(G_k)$, since adjacency in G_k would imply adjacency in $\tilde{G}$. Hence, $(S, k) \in X$.

Conversely, let $(S, k) \in X$ and define $\tilde{S} = \{(v, k) : v \in S\}$. Then $\tilde{S} \subseteq \tilde{V}$, and $\tilde{S} \in \mathscr{S}(\tilde{G})$, since adjacency in $\tilde{G}$ would imply adjacency in G_k, contradicting the stability of S.

Thus, there is a bijection between the variables $x_{\tilde{S}}$ in DPCP-SCF and the variables x_S^k in WLCP-SC. Moreover, under this correspondence, the condition $\tilde{S} \cap P_v \neq \emptyset$ is equivalent to $v \in S$, so constraints (2) correspond to constraints (1) of WLCP-SC. Similarly, the condition $\tilde{S} \cap Q_k \neq \emptyset$ is equivalent to $S \in \mathscr{S}(G_k)$, and since $m(k) = 1$, constraints (3) correspond to constraints (2) of WLCP-SC.

Finally, when all color weights in WLCP-SC are equal to one, the objective functions of the two formulations coincide. □

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the authors used ChatGPT and Claude in order to improve language and readability. After using this tool/service, the authors reviewed and edited the content as needed and take full responsibility for the content of the published article.

References

Bhyravarapu, S., Hartmann, T. A., Hoang, H. P., Kalyanasundaram, S., & Vinod Reddy, I. (2024). Conflict-free coloring: Graphs of bounded clique-width and intersection graphs. *Algorithmica*, *86*, 2250–2288. doi:10.1007/s00453-024-01227-2.

Bhyravarapu, S., Kalyanasundaram, S., & Mathew, R. (2022). Conflict-free coloring on claw-free graphs and interval graphs. In *47th International Symposium on Mathematical Foundations of Computer Science (MFCS 2022)* (pp. 19:1–19:15). Schloss Dagstuhl–Leibniz-Zentrum für Informatik. doi:10.4230/LIPIcs.MFCS.2022.19.

Brélaz, D. (1979). New methods to color the vertices of a graph. *Communications of the ACM*, *22*, 251–256. doi:10.1145/359094.359101.

Chen, K., Kaplan, H., & Sharir, M. (2009). Online conflict-free coloring for halfplanes, congruent disks, and axis-parallel rectangles. *ACM Transactions on Algorithms (TALG)*, *5*, 1–24. doi:10.1145/1497290.1497292.

Desrosiers, J., Lübbecke, M., Desaulniers, G., & Gauthier, J. B. (2026). *Branch-and-Price*. Springer Cham. doi:10.1007/978-3-031-96917-1.

Even, G., Lotker, Z., Ron, D., & Smorodinsky, S. (2003). Conflict-free colorings of simple geometric regions with applications to frequency assignment in cellular networks. *SIAM J. Comput.*, *33*, 94–136. doi:10.1137/S0097539702431840.

Fabrici, I., Lužar, B., Rindošová, S., & Soták, R. (2023). Proper conflict-free and unique-maximum colorings of planar graphs with respect to neighborhoods. *Discrete Applied Mathematics*, *324*, 80–92. doi:10.1016/j.dam.2022.09.011.

Feo, T. A., & Resende, M. G. C. (1995). Greedy randomized adaptive search procedures. *Journal of Global Optimization*, *6*, 109–133. doi:10.1007/BF01096763.

Furini, F., Malaguti, E., & Santini, A. (2018). An exact algorithm for the partition coloring problem. *Computers & Operations Research*, *92*, 170–181. doi:10.1016/j.cor.2017.12.019.

Golovach, P. A., Johnson, M., Paulusma, D., & Song, J. (2017). A survey on the computational complexity of coloring graphs with forbidden subgraphs. *Journal of Graph Theory*, *84*, 331–363. doi:10.1002/jgt.22028.

Gupta, S., & Mathew, R. (2026). Bounds and hardness results for conflict-free choosability. In *International Conference on Current Trends in Theory and Practice of Computer Science* (pp. 461–474). Springer.

Har-Peled, S., & Smorodinsky, S. (2003). On conflict-free coloring of points and simple regions in the plane. In *Proceedings of the nineteenth annual symposium on Computational geometry* (pp. 114–123). doi:10.1145/777792.777810.

Held, S., Cook, W., & Sewell, E. C. (2012). Maximum-weight stable sets and safe lower bounds for graph coloring. *Mathematical Programming Computation*, *4*, 363–381. doi:10.1007/s12532-012-0042-3.

Jensen, T. R., & Toft, B. (2011). *Graph coloring problems*. John Wiley & Sons. doi:10.1002/9781118032497.

Li, G., & Simha, R. (2000). The partition coloring problem and its application to wavelength routing and assignment. In *Proceedings of the First Workshop on Optical Networks*.

Lucci, M., Nasini, G., & Severín, D. (2024). Solving the list coloring problem through a branch-and-price algorithm. *European Journal of Operational Research*, *315*, 899–912. doi:10.1016/j.ejor.2024.01.038.

Lucci, M., Nasini, G., Tolomei, P., & Torres, L. M. (2025). New framework for conflict-free coloring of hypergraphs and other graph coloring problems. *Procedia Computer Science*, *273*, 357–364. doi:10.1016/j.procs.2025.10.319.

Malaguti, E., Monaci, M., & Toth, P. (2011). An exact approach for the vertex coloring problem. *Discrete Optimization*, *8*, 174–190. doi:10.1016/j.disopt.2010.07.005.

Malaguti, E., & Toth, P. (2010). A survey on vertex coloring problems. *International transactions in operational research*, *17*, 1–34. doi:10.1111/j.1475-3995.2009.00696.x.

Mehrotra, A., & Trick, M. A. (1996). A column generation approach for graph coloring. *INFORMS Journal on Computing*, *8*, 344–354. doi:10.1287/ijoc.8.4.344.

Pach, J., & Tardos, G. (2009). Conflict-free colourings of graphs and hypergraphs. *Combinatorics, Probability and Computing*, *18*, 819–834. doi:10.1017/S0963548309990290.

Pach, J., & Tardos, G. (2010). Coloring axis-parallel rectangles. *Journal of Combinatorial Theory, Series A*, *117*, 776–782. doi:https://doi.org/10.1016/j.jcta.2009.04.007.

Smorodinsky, S. (2013). Conflict-free coloring and its applications. In *Geometry—Intuitive, Discrete, and Convex: A Tribute to László Fejes Tóth* (pp. 331–389). Springer.